

Correlation Aware Dynamic Solution for Merging Small Files based on Similarity and Clustering in HDFS

Hanène Chettaoui

hanene.chettaoui@isamm.uma.tn

Tunis El Manar University

Farah Hkiri

Tunis El Manar University

Research Article

Keywords: Big Data, Hadoop, HDFS, Small File, Correlation, Similarity, Clustering

Posted Date: August 27th, 2025

DOI: <https://doi.org/10.21203/rs.3.rs-6944075/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

Correlation Aware Dynamic Solution for Merging Small Files based on Similarity and Clustering in HDFS*

Hanène Chettaoui and Farah Hkiri

Abstract In distributed processing environments such as Hadoop, Hadoop Distributed File System (HDFS) is a popular open-source solution for storing and managing a massive number of files. HDFS faces several issues when it comes to handling a large amount of small files. To overcome this drawback, we propose a new dynamic strategy CHAC (for Correlation and Hierarchical Ascending Clustering) dealing with the small files problem. The major contribution of our strategy is that considering correlations between small files is the criterion used in the merging process into large files. For this purpose, several criteria such as file size, requests number and requesting clients are taken into account through analyzing the user access pattern. In the current version of our proposal, a formula inspired from information retrieval is used to quantify the weight of each small file. Moreover, a Cosine similarity-based method coupled with a hierarchical ascending clustering algorithm is used as a shrewd grouping tool of correlated small files. To demonstrate the effectiveness of CHAC, a series of experiments were performed over ten distinct datasets. Compared with other solutions, the obtained results highlight that our proposal offers interesting performances for reducing the number of obtained large files as well as the NameNode memory consumption. CHAC also optimizes the use of the DataNodes storage space by increasing the average disk utilization of data blocks. Moreover, it reduces the storage time taken to store large files. As our solution is based on file correlations, an in-depth analysis of the quality of the obtained large files is carried out highlighting the effectiveness of CHAC.

Key words: Big Data; Hadoop; HDFS; Small File; Correlation; Similarity; Clustering

1 Introduction

1.1 Background and Motivations

The "Big Data" term is used to refer to a collection of large datasets, which is difficult to process using traditional data processing tools. Data can be categorized into three main types: structured, unstructured, and semi-structured. As vast amounts of data continue to grow exponentially, it must be processed, stored, and analyzed for diverse

* This paper is a largely extended version of the work presented in [22].

Hanène Chettaoui^{1,2}

¹LIPAH Laboratory, Faculty of Sciences of Tunis, Tunis El Manar University, Tunisia

²Higher Institute of Multimedia Arts of Manouba, University of La Manouba, Tunisia

e-mail: hanene.chettaoui@isamm.uma.tn

Farah Hkiri¹

¹LIPAH Laboratory, Faculty of Sciences of Tunis, Tunis El Manar University, Tunisia

e-mail: farah.hkiri@etudiant-fst.utm.tn

purposes. The concept of Big Data is often described by the five V's [38], namely Volume, Variety, Velocity, Veracity, and Value.

Big Data presents significant opportunities in various fields, including healthcare, supply chain and logistics, marketing and advertising, smart cities, media and entertainment, cybersecurity, natural disasters, and education [14]. Hadoop is a widely used open-source framework for managing Big Data, consisting of two main components: HDFS (Hadoop Distributed File System) and MapReduce. HDFS is composed of three components: NameNode, DataNodes, and clients [28, 66]. The NameNode is responsible for managing the metadata of files stored in the DataNodes, while the DataNodes store files on disk, dividing them into data blocks of a predefined size. Clients, in turn, request the stored files. Files are categorized as either large or small based on their size. In the context of Hadoop, a small file refers to a file whose size is smaller than the data block size [32]. HDFS is very efficient when storing and processing large files [26]. However, when dealing with a large number of small files, HDFS encounters what is commonly known as the Small Files Problem [16, 45].

Several applications [6, 34] generate large volumes of small files, including those based on machine learning and deep learning training [46, 83], Internet of Things (IoT) [54, 64], data lakes [31], data management [35] and e-governance systems [37]. In these contexts, managing large amount of small files can significantly degrade access, processing and data transfer performances [41, 42, 54, 57].

The difficulties of HDFS dealing with a large quantity of small files arise mainly due to the following reasons: (i) Metadata overhead at the NameNode: HDFS allocates a full data block for each small file, regardless of its size. When handling a large number of small files, the NameNode becomes overloaded with metadata, as it must maintain information about each file [50]. On average, each file's metadata occupies 150 bytes in the NameNode's memory. Given the memory limitations of the NameNode, this leads to performance degradation. (ii) Inefficient storage on DataNodes: each small file is stored in a separate data block, leading to increased disk space overhead and additional I/O operations. (iii) Reduced performance in MapReduce jobs: when processing many small files, MapReduce jobs require a large number of seek operations and thus consuming extra bandwidth for performing I/O.

1.2 Generic Proposal and Contributions

In the literature, several works have been proposed to address the small files problem. Most of these focus on merging small files into larger ones, mainly based on file size. However, only some work give attention to the intrinsic relationships between files. In this context, it is crucial to incorporate additional criteria to determine which small files should be merged into the same large file, taking into consideration the user access pattern.

Figure 1 illustrates the generic process of our proposal for file merging. The flowchart begins with the pre-processing of access pattern to generate a Frequency Matrix for small files. This matrix assess for each small file its request frequency by each client. It is then used to produce the Weight Matrix assessing the importance of each file. Next, a Similarity Matrix is computed using a selected correlation measure. This matrix serves as input for an algorithm having for purpose to iteratively construct groups of correlated small files. The Similarity Matrix is then updated according to the new obtained groups. A stopping criterion is evaluated indicating whether there further groups to determine. If it is not satisfied, the grouping step is repeated. Otherwise, once the final groups are obtained, the small files within each group are merged into a large file. Finally, the merged files are stored in the DataNodes, and an index file containing metadata is updated in the NameNode.

In this regard, as an instance of our generic proposal, a novel strategy is designed for addressing the small files problem, called CHAC (Correlation and Hierarchical Ascending Clustering). CHAC addresses the small files problem by merging multiple files into larger ones based on their correlation. The core assumption is that files with high correlation are frequently accessed together by clients. This correlation-aware merging strategy reduces the management costs of small files and significantly enhances I/O performance, improving then access efficiency of small files and by the way energy consumption. Indeed, our main focus is not only reducing the metadata storage requirement of the NameNode and the disk utilization of DataNodes, but also finding an interesting trade-off with obtained large file quality through correlation assessment.

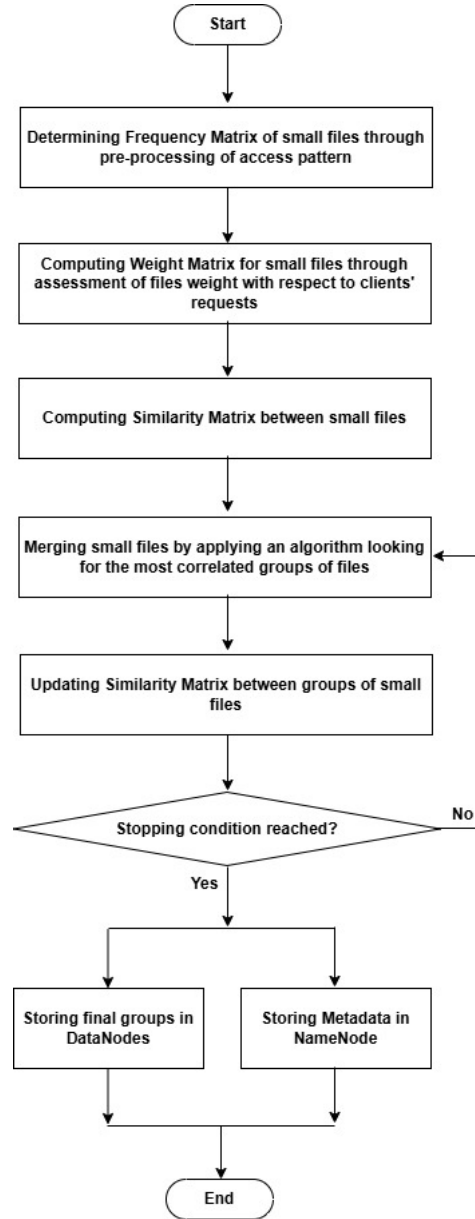


Fig. 1: Flowchart of our general proposal

In this context, in order to assess the importance of each file, several parameters are considered, namely the size, the number of the requesting clients as well as the request frequency by each client. In our generic process, the weight of each file can be computed using different ways. In this work, CHAC is proposed with a formula inspired by the weighting technique *tf-idf* from information retrieval [33, 60].

For determining file correlation, several metrics can be adopted. In its current version, CHAC adopts the Cosine similarity measure. The Cosine similarity is a useful measure for the evaluation of the correlation between files, represented through their associated weight vectors, by calculating the angle between them. It is robust to varying vector lengths and its geometric interpretation enhances its utility, contributing to its prevalence in diverse fields. It is indeed among the most popular measure in the inner product family [15, 53, 56].

Based on these features and as an efficient grouping tool, a hierarchical ascending clustering algorithm [58, 67] is applied to group similar files into the same cluster. An important advantage of hierarchical clustering is that there is no need for beforehand identifying the number of clusters. When applying the clustering algorithm in CHAC, it finds the adequate number of clusters according to some considered constraints and stopping conditions. Each resulting cluster corresponds to an obtained large file by merging the associated correlated small files. This clustering process enables CHAC to significantly reduce the metadata storage required to manage a large number of small files by minimizing the number of obtained clusters. By consolidating correlated files, the system improves both storage and access efficiency, thereby optimizing HDFS I/O performance. Indeed, if some file has been accessed, then the remaining files in the same cluster are the most likely to be accessed as well.

The main contributions of this paper are as follows:

- Firstly, we survey in detail the main solutions dealing with the small file problem, and more specifically, those based on file merging. Compared to [22], we add several other strategies. Furthermore, a largely extended discussion of the surveyed strategies is given in the present work. Some open challenges and issues in the field of small file problem are then clearly identified and discussed. This will allow us to highlight the limitations of the solutions of the literature motivating our proposal.
- Secondly, we present our dynamic strategy taking into account file correlations using several criterion. In this respect, a similarity-based process relying on the Cosine measure is then adopted. Then, an algorithm for hierarchical ascending clustering is used to find groups of correlated small files. Each group is merged into a large file. In comparison to the conference version, further details are given to describe each step of the proposed strategy. In addition, a complete running example is added to illustrate how the strategy runs.
- Thirdly, we conduct an in-depth comparison with other solutions of the literature. In this article, we propose to extend our previous work by offering a more complete comparison process and further details on obtained results, as follows:
 - Use of a larger set of data – ten distinct datasets – with a specific file distribution and more details on associated small files.
 - Study of the impact of different parameters to consider on performance evaluation results.
 - Focus on a comparison with the optimal number of data blocks as well as a study of the average disk utilization when dealing with DataNodes storage performance.
 - More details on the metadata consumption of the NameNode.
 - Considering the storage time performance.
- Fourthly, an in-depth study of the quality of the obtained large files after small file merging from a compactness point of view. Two frequently used measures for clustering quality evaluation are exploited to this purpose.

1.3 Paper Organization

The remainder of the paper is organized as follows. The next section depicts the main related work and offers a discussion of the reviewed solutions. Then, the proposed CHAC strategy is described in detail in Section 3. Section 4 illustrates CHAC through a complete example. In Section 5, the obtained experimental results are analyzed and discussed from different perspectives. Section 6 evaluates the quality of the obtained large files. Finally, a conclusion and some suggestions for future work are given.

2 Related Work

Several works are proposed dealing with the HDFS inefficiency in handling small files. They can be classified into three main categories based on the way they deal with small files, as follows [79]:

- *Combining small files into large files*: this category consists of merging small files into large files in order to reduce the amount of metadata needed for their storage.
- *Specialized Distributed File System (DFS) for small files*: architectural changes in Hadoop are proposed by building some DFS specialized only in treating small files [8, 27, 40, 71].
- *Improving processing framework*: associated works build a better accessing and processing framework for small files [23, 51, 55, 76].

As our strategy belongs to the first category, we mainly concentrate on related work focusing on merging small files, classified into four main classes:

- *Merging solutions provided by Hadoop*: Hadoop offers some solutions to merge small files into large files.
- *Merging files based on indexation*: this class consists of merging small files into larger ones and maintaining an efficient index corresponding to these merged files.
- *Merging files based on cache*: the caching mechanism is used to efficiently load the merged files or the metadata of the merged files.
- *Merging files based on files correlation*: correlated small files are merged into the same large files.

Hereafter, we review the main research work in each class in ascending order of the publication year. We will also discuss some other existing solutions not belonging to any of the classes mentioned above.

2.1 Hadoop Solutions and Associated Improvements

Hadoop proposes three solutions, namely Hadoop Archive (HAR) [2], SequenceFile [4] and MapFile [3]. HAR is a file archiving tool, which packs multiple small files into a single HAR file with a two-level index. However, these archives are immutable. SequenceFile is a flat file storing data in the form of key-value pairs. Keys hold files names while values store the files contents. However, the entire SequenceFile must be loaded in the memory when reading a file. MapFile is a sorted version of SequenceFile. It generates an additional index file to enable faster key search. However, the entire index must be loaded in the memory, limiting therefore the number of files MapFile can hold.

Vorapongkitipun and Nupairoj propose an improvement of existing HAR called New Hadoop Archive (NHAR) [68]. Instead of using two level indexes, NHAR employs a single-level index. Instead of using master-index approach, NHAR creates a hash table of index information and split information over multiple index files. The hash code file is generated from the file name and is modded with the number of index files to determine which index file holds the metadata.

On their side, in [25], Dev and Patgiri propose an improvement of existing HAR called Hadoop ARchive Plus (HAR+). Instead of using one NameNode for storing the metadata, HAR+ uses multiple NameNodes. HAR+ uses a single level index structure and a hashing function. For each file, a hash code is generated based on the filename. The obtained code is used to identify the appropriate NameNode, including a Hashtable containing files metadata.

Renner *et al.* propose an extension of HAR, denoted AHAR (Appendable Hadoop Archives) [61], by adding appendable file format to solve the small file problem. A new small file is added to one of the multiple part files, which contain the data of the already archived files, using the first fit algorithm. Thus, the index and master index files are updated and rewritten to HDFS. A Red-Black tree structure is used for indexing towards efficient lookup.

It is important to note that other contributions exist in the literature aiming at further improving HAR as well as the other Hadoop-based solutions, namely SequenceFile and MapFile.

2.2 Indexation-based Solutions

In [50], Mackey *et al.* propose a new strategy to store the metadata efficiency based on the assumption that each client is assigned a quota in the file system, for both the space and number of files. Small files are merged into archiving groups. The proposed strategy is based on archives metadata and metadata index. An archive metadata is

created for each archive and contains information about archive files. The index serves as a meta-meta data layer for the data in the archive.

To deal with the small files problem, Fu *et al.* present a new strategy called Flat Lightweight File System (iFlatLFS) [29]. iFlatLFS uses a simple file metadata scheme. It combines file metadata, such as file ID, file type, creation time, checksum, etc., into the file header and stores it together with the file data. Thus, each file consists of a header and its data. Each metadata area is a sequence of index slots. A slot represents a small file stored in the corresponding data area. Each index item contains the file size and the file physical address.

Zhu *et al.* propose a strategy for the small files problem called PHDFS (Pile-HDFS) [83] for deep learning applications. PHDFS merges small files into large files based on file size. The merged small files are stored in a pile which is a file aggregation structure containing a pile state and the content of small files. A pile index is created which records the file name to the merged file name and the pile identifier mapping.

In [79], Zhai *et al.* propose an index-based archive called Hadoop Perfect File (HPF). HPF tries to merge small files into large files using two hash functions, namely EHF (Extendible Hashing Function) and OPMPHF (Order Preserving Minimal Perfect Hash Function) to store metadata in different index files.

Chen *et al.* propose a novel archive system called SFM (Small File Merger) [20]. SFM uses a two-level index and a merged binary file. The two-level index is composed of an index and a master index. The index contains several index entries where each entry contains four parts: a series of key-value (the key is filename and the value is file's metadata), BloomFilter, Trailer, and a one-byte Flag. The master index contains each index entry's minKey, maxKey and position. SFM provides the capability of editing archived files directly without the need of re-archiving. Indeed, once a new index entry is appended in the index, the master index is also increased with the addition of a new record at the end.

Wu *et al.* propose a strategy to deal with small files problem using a pile structure [73]. The set of small files are merged in several piles according to their sizes using the worst-fit strategy. Each pile contains a pile identifier, an index list and small files contents. The index list stores a multiple single files indexes where each one includes the pile identifier, the file name, the offset and the file size. After the generation of piles set, the small files in each pile are merged in a sequence file and a compression strategy is used to reduce the sequence file size. A global index is created and records the index list.

Zhang *et al.* propose a small file-merging strategy called BTDFS (B+ Tree-based Distributed File Storing mechanism) [81]. Files are classified according to their size into three categories: large files, small files and ultra-small files. A large file is uploaded directly to the HDFS. An ultra-small file is stored directly in the index. On their side, small files are merged into large files one by one and then added to the index queue to form an index file. Each index called bucket is organized into B+ tree for storage. The data in buckets is saved into OS pages. Four types of pages are used, namely meta page, freelist page, branch page and leaf page.

Alange and Mathur propose a virtual file system layer called VFS-HDFS. In their work, small files are classified into several categories according to their content and their attributes [10]. VFS-HDFS uses a root file table, a file table for each category (called bucket) and a container file for each category. This work is next applied on a corpus of British library text files in [11].

In [63], Sharma *et al.* propose a strategy called HB-EHA (Hash Based-Extended Hadoop Archive). HB-EHA creates Hadoop archive from the small files and then uses two special hash functions, namely SSHF (Scalable-Splittable Hash Function) and HT-MMPHF (Hollow Trie Monotone Minimal Perfect Hash Function). The first function is used to merge small files by distributing the archives metadata to the associated slave index files. HT-MMPHF is responsible for sorting metadata at slave index files. The obtained slave index files are then turned into final index.

Zhang and Chen propose a strategy to treat the small files problem in multi-modal campus one-stop data [80]. Small files are merged according to their sizes and the index files are constructed using an extensible hash function and MWHC (from Majewski, Wormald, Havas and Czech) hash function to establish the mapping between the small file and the merged file. The extensible hash function generates an index file in the form of array storing the bucket pointer directory. The files in each bucket are sorted first to determine the index position of the files in each bucket. Then, the MWHC hash function generates an index location information.

In [49], Lui *et al.* propose an archive-based strategy to merge small files. This strategy is based on two indexes and uses an extendible hashing function for the names of small files in order to put them in several buckets. A minimal perfect hash function is also relied on to locate the position of the desired metadata of a small file.

2.3 Cache-based Solutions

In [17], Bok *et al.* propose a distributed caching scheme to efficiently access small files. The proposed scheme merges small files into large files, denoted as combined files. The block metadata of the combined file is stored in the NameNode. The frequently requested small files are stored in the cache DataNode. The client cache maintains cache metadata in addition to small files and block metadata that the client frequently requested. The cache metadata shows location information of small files maintained in each node cache.

In [70], Wang *et al.* propose a strategy for merging small into large files called Multilevel Optimization Storage Method (MOSM) based on a prefetching cache. Small files are merged into large files by the equilibrium merger queue algorithm. After merging, a hybrid index is created including a global index and a merged file index. The global index records the correspondence between the small file and the merged file in the form of data structure named File Name Index Tree. The merged file index uses B+ tree to store the small file names and the corresponding synchronization markers for each merged file. On the one hand, an index prefetching mechanism is triggered if the client needs to read a small file and there is no corresponding index in the local cache pool. The merged file index and metadata are hence added in the client cache. On the other hand, a merged file prefetching mechanism is triggered if the client intends to read a small file and has the index cache of the file. The merged file is then added in the client cache.

In [64], Siddiqui *et al.* propose a strategy dealing with the IoT small files problem called PSFMF (Pseudo-cache-based Small Files Management Framework). PSFMF divides IoT small files into two parts, namely a metadata (MD) which is stored into the Index of File (IoF) cache and a file's body into Cache of File (CoF). Small files are added according to their sizes in the one of the predefined four queues. Therefore, the metadata of each small file is added to the corresponding metadata file. PSFMF associates the metadata file name to an Accessibility Index (AI) and produces a number of links.

Gao *et al.* propose a distributed cache mechanism to deal with the small files problem for Deep Reinforcement Learning [30]. Small files are merged in large files according to their sizes. As a file is stored on three different DataNodes, all the DataNodes containing a requested small files load the large file in their cache. This strategy reduces the time of Deep Reinforcement Learning spent by reading dataset from cache.

In [75], Xu *et al.* present a strategy that merges small files into large ones and uses a two-level collaborative caching strategy. The merging process is based on files size and using several algorithms such as genetic algorithms, particle swarm algorithms, descending first-fit algorithms, and descending best-fit algorithms. This strategy uses a two-level collaborative caching strategy that is to say that the server caches the small file data and the client caches the index information of the small file (file name, file size, name of the merged file, and starting position). For the caching replacement problem, the server side adopts a replacement strategy based on recent access frequency, while the client side adopts the Adaptive Replacement Cache (ARC) strategy proposed by IBM.

Ishengoma proposes a strategy dealing with the small files problem, called Efficient Small File Management Algorithm (ESFMA), for enhanced e-government services [37]. ESFMA adopts a mechanism of block aggregation in order to group small files into large files based on files size. A hierarchical metadata structure distributes the massive number of files metadata into different layers. The files metadata, frequently having to have read and write accesses, are added to the NameNode cache. The cache process is based on a cache-hit ratio. This latter is a function of cache size and total metadata size, taking into account a cache replacement policy.

2.4 Correlation-based Solutions

In [82], Zhang *et al.* present a merging strategy in a rendering system based on Particle Swarm Optimization (PSO) and Support Vector Machine (SVM). Indeed, rendering is the process of generating a final image or animation from a 2D or 3D model. The proposed strategy tries to merge small RIB files in the same scene without building index files by benefiting from the characteristic of RIB file. PSO selects the optimal merge values for multiple sets of scenes. The optimal merge value ensures the shortest rendering time for a scene. SVM generates a general SVM model, which can be used to get the optimal merge by learning the samples generated by PSO.

Cai *et al.* [18] propose a merging strategy based on files distribution and files correlation. File correlation is determined from the history of files access log, while considering three weights representing values of different levels of correlation. No further details on how correlation is computed were given.

Wai *et al.* present in [69] a strategy, denoted MAGC, for dealing with the small files problem using agglomerative hierarchical clustering. The proposed strategy only takes into account the files size during the merging step. The Euclidean distance is used to calculate the distance matrix between all files.

In [59], a strategy called CSFC (Clustering Small Files based on Centroid) is presented. In CSFC, a small file is added to the cluster if its distance to the cluster centroid is less than a predefined threshold, while maintaining the cluster size smaller than the block size.

In order to deal with numerous small text files containing agricultural resources, a dedicated strategy is described in [48]. In the agricultural operation process, there are correlations between files, which are the precursor and subsequent relations. Relying on the terms contained in files, correlation is computed using the vector space model and is used to identify the small files to be combined into large files.

In [21], Chen *et al.* propose a storage architecture called HDFSA (Hadoop Distributed File System based on Association rule mining). HDFSA applies a pattern-mining algorithm, namely FP-Growth, to determine the relationship between small files according to clients IP, the access time and files paths. The merged small files are stored into SequenceFile files.

In [72], Wang *et al.* present a consolidation schema based on graph clustering to deal with the small files problem for industrial applications. The small file consolidation problem is modeled as a graph-clustering problem where nodes represent files and edges represent the queries number requiring the two files linked by the edge. The graph is generated from a workload. The hierarchical clustering algorithm is then used to merge nodes into single file where its size does not exceed the block size.

2.5 Other Solutions

In [34], He *et al.* propose a strategy called Tetris Merge (TM) for merging small files based on balance of data block. The main idea is to distribute the small files into large files relying a queue-based method. File merging queues are used to store the file collection to be merged. A small file is added to the file queue with the maximum free space.

In [7], Ahad and Biswas propose a strategy called Dynamic Merging based Small Files Storage (DM-SFS). Relying on a partition policy, DM-SFS aims to merge small files into several large files according to two parameters, namely file type and file size using Next-Fit allocation policy. The dynamic Variable-Size partitioning is applied for the memory blocks allocation. The obtained large files are then stored in MapFile files.

In [39], Jayalakshmi and Srinivasan design a file-merging algorithm that can be run when a large file is to be written in the HDFS or once every epoch. A greedy knapsack problem is used to add many small files into the remaining data block size containing a large file. The objective is to minimize the amount of unfilled space at the end of each block.

In [65], a strategy is presented to group small files into four categories, which are serious reading, focused composing, intensive reading and composing, and lean reading and composing. After grouping, the algorithm tries to merge files from the first three categories into the fourth category. If some files do not fit into huge documents, they remain as small files.

In [12], Ali *et al.* propose a merging strategy for small files after a simple data cleansing. Indeed, data cleansing aims to make sure that all the unnecessary information is removed from all provided files. The obtained files are grouped according to the file type, for example, text files of different types such as `txt`, `csv`, `xlsx`, etc. Then, the Best Fit algorithm is used to merge the obtained files into large files. The authors also present in [13] a strategy for the small files problem called EBFM (Enhanced Best Fit Merging). EBFM merges small files into large ones based on file format and size criteria using Best Fit algorithm. All files are scanned to select the small files according to a threshold value point. Small files are categorized according to the file type and merged according to the file size.

In [9], Al-Masadeh *et al.* propose a strategy called Tiny Datablock-HDFS (TD-HDFS) to deal with the small files problem. The principle of TD-HDFS is to change the default block size to set it equal to 5 MB. Once these

tiny data blocks are allocated to the new small files, they are merged on a master data block with only one metadata file. The size of the master data block does not exceed the default block size.

2.6 Discussion

In our analysis of the literature, reviewed strategies focusing on merging small files are classified into four main classes. The strategy we propose in this work deals with the small files problem by further looking for a deeper assessment of file correlation. Our purpose is to offer a finer selection of small files to be merged in the same large file.

In this respect, Table 1 summarizes existing work dealing with the small files problem through files correlation. The main issues addressed are as follows: (i) Used methods, (ii) Parameters for the merging process, (iii) Metrics for experimental evaluation.

Reference	Outcome of the paper	Used methods	Parameters for the merging process	Metrics for experimental evaluation
[82]	Merging strategy in a rendering system	Particle Swarm Optimization, Support Vector Machine	File size	Storage time, NameNode memory consumption, Rendering time
[18]	Merging strategy based on files distribution and files correlation	Using of merge queues	File size, Files access, Correlation levels (no further details on how correlation is computed were given)	Time consumption of importing files, NameNode memory consumption, Time consumption of file access
[69]	Merging strategy based on clustering mechanism	Agglomerative Hierarchical Clustering, Euclidean distance	File size	NameNode memory consumption
[59]	Merging strategy based on clustering mechanism	Cluster centroid	File size	Average of Data Difference Ratio (DDR), Time consumption for generating clusters
[48]	Merging strategy for the agricultural operation process	Vector Space Model	File size, Precursor and subsequent relations between agricultural files	NameNode memory consumption, Average reading time consumption
[21]	Merging strategy based on association rule mining	FP-Growth	File size, Clients IP, Access time, Files paths	NameNode memory consumption, File upload time, File download time
[72]	Merging strategy based on graph clustering	Graph clustering	Queries number, File size	Time consumption of generating the consolidation schemas, File upload time, Total time consumption of both generating the consolidation schema and transmitting the requested files

Table 1: A quick review to the existing strategies based on file correlation for handling massive small files

Most of the existing approaches only rely on file size as a base for guessing correlation between files. They neither consider user access pattern or application contexts nor content characteristics and semantic relationship. We remark that all the existing strategies take into account files size in the merging process. Several techniques are used which highlights the diversity of methods. The NameNode memory consumption is the most used metric for experimental evaluation. Indeed, the NameNode memory consumption is inevitable due to design of HDFS. However, user access pattern is not considered during the merging process. Thus, the file correlation ignores how the files are used. However, in several applications, such as recommendations based on user comments [37], it is necessary to co-locate user comments related to the characteristics of a specific product in the same blocks for

application speedup.

In the next section, we propose a new strategy dealing with the small files problem while considering user access pattern. Several parameters like requests number and requesting clients are hence taken into account, while using of course on file sizes. The main objective is to determine the most correlated files in order to merge them in the same large file by respecting the constraint that the large file size does not exceed the block size. Identifying correlations enables faster and more efficient data processing by grouping together small files that are frequently accessed simultaneously by clients. Our strategy also aims to minimize the NameNode memory consumption.

3 Proposed Strategy

We now introduce our proposed approach, CHAC which is a dynamic strategy specifically designed to address the small files problem. A file is classified as small if its size is less than a data block. Otherwise, it is considered as a large file. CHAC operates by identifying correlations and grouping small files using hierarchical ascending clustering. The main objective of CHAC is to merge as many correlated small files as possible into a single large file, without exceeding the size of a data block. This significantly reduces the number of resulting large files generated from a huge number of small files, while preserving the quality of the merging process. CHAC evaluates the relationships between small files by analyzing their access pattern history. File correlation is computed based on several criteria, namely file size, requests number and the clients making the file requests.

CHAC aims to reduce the memory consumption of the NameNode and the number of files stored in DataNodes. It is composed of five steps as follows:

- Step 1:* Generation of the frequency matrix
- Step 2:* Generation of the weight matrix
- Step 3:* Generation of the similarity matrix
- Step 4:* Merge of the correlated small files
- Step 5:* Storage of the obtained large files in the HDFS

Before delving into the steps of our strategy, we list in Table 2 the different notations used in CHAC.

Notation	Description
FM	Frequency matrix
WM	Weight matrix
SM	Similarity matrix
$TotalClients$	Total number of requesting clients
F_i	File with index i
Cl_j	Client with index j
$Size(F_i)$	Size of F_i
$RequestsNb(F_i, Cl_j)$	Requests number for F_i by Cl_j
$TotalClients(F_i)$	Total number of clients requesting F_i
$Weight(F_i, Cl_j)$	Weight of F_i on Cl_j
$\vec{F}_i$	Weight vector associated to F_i
$Similarity(F_i, F_j)$	Similarity between F_i and F_j
C_k	Cluster with index k

Table 2: Notations used in CHAC

3.1 Frequency Matrix Generation

The first step of our strategy involves constructing the frequency matrix FM , where the rows represent the set of n small files $\{F_1, F_2, \dots, F_n\}$, and the columns represent the set of m requesting clients $\{Cl_1, Cl_2, \dots, Cl_m\}$. Each cell in FM indicates the requests number, denoted $RequestsNb(F_i, Cl_j)$, made by a client Cl_j for a file F_i . The matrix FM is generated by collecting and analyzing data access logs, such as *NameNode.log* and *DataNode.log*. The structure of FM is illustrated in Figure 2.

$$FM = \begin{pmatrix} RequestsNb(F_1, Cl_1) & RequestsNb(F_1, Cl_2) & \dots & RequestsNb(F_1, Cl_m) \\ RequestsNb(F_2, Cl_1) & RequestsNb(F_2, Cl_2) & \dots & RequestsNb(F_2, Cl_m) \\ \vdots & \vdots & \ddots & \vdots \\ RequestsNb(F_n, Cl_1) & RequestsNb(F_n, Cl_2) & \dots & RequestsNb(F_n, Cl_m) \end{pmatrix}$$

Fig. 2: Structure of FM

3.2 Weight Matrix Generation

The second step involves constructing the weight matrix WM , where the rows represent the set of small files and the columns correspond to the set of requesting clients. The weight vector $\vec{F}_i$ associated to each small file F_i considering client requests is then computed from the frequency matrix, as follows:

$$\vec{F}_i = \begin{pmatrix} Weight(F_i, Cl_1) \\ Weight(F_i, Cl_2) \\ \dots \\ Weight(F_i, Cl_m) \end{pmatrix}$$

Motivated by techniques from the field of information retrieval [33, 60], the weight assigned to a file F_i on a client Cl_j is computed using Formula 1, where $RequestsNb(F_i, Cl_j)$ reflects the importance of the file F_i in the client Cl_j . In other words, the more F_i is requested by Cl_j , the greater it is important. The logarithmic component captures the relevance of F_i compared to other files. It indicates the ratio of the number of requesting clients of the file F_i , denoted $TotalClients(F_i)$, with respect to the total number of requesting clients, denoted $TotalClients$. Since $\log(1)$ is equal to 0, which arises in the case where a file F_i is requested by all requesting clients, 1 is added to only take account of its number of requests by the client Cl_j in the weight computation.

$$Weight(F_i, Cl_j) = RequestsNb(F_i, Cl_j) \times \left(\log \left(\frac{TotalClients}{TotalClients(F_i)} \right) + 1 \right) \quad (1)$$

A large value of $Weight(F_i, Cl_j)$ signifies that F_i is important for Cl_j and is specific relative to other files.

The weight matrix is represented in the form shown in Figure 3. Algorithm 1 describes the generation process of the weight matrix.

$$WM = \begin{pmatrix} Weight(F_1, Cl_1) & Weight(F_1, Cl_2) & \dots & Weight(F_1, Cl_m) \\ Weight(F_2, Cl_1) & Weight(F_2, Cl_2) & \dots & Weight(F_2, Cl_m) \\ \vdots & \vdots & \ddots & \vdots \\ Weight(F_n, Cl_1) & Weight(F_n, Cl_2) & \dots & Weight(F_n, Cl_m) \end{pmatrix}$$

Fig. 3: Structure of WM

Algorithm 1: Generation process of the weight matrix WM**Data:** Frequency matrix FM of size $n \times m$ where n is the total number of files and m is the total number of clients.**Result:** Weight matrix WM**Begin**

```

1 TotalClients  $\leftarrow m$ 
2 for each file  $F_i$  in FM do
3    $TotalClients(F_i) = |\{v \text{ in } FM[i] \mid v > 0\}|$ 
4   for each requesting client  $Cl_j$  in FM do
5      $WM[i][j] \leftarrow FM[i][j] \times \left( \log \left( \frac{TotalClients}{TotalClients(F_i)} \right) + 1 \right);$ 
End

```

3.3 Similarity Matrix Generation

The following step entails constructing the similarity matrix SM by calculating the similarity value between each pair of files F_i and F_j . For this purpose, in the current version of CHAC, the Cosine similarity is adopted. This choice is motivated by the several advantages of this measure [15, 53, 56]. The construction of the similarity matrix includes several sub-steps:

- **Checking the size constraint:** Before computing the similarity value between F_i and F_j , it is necessary to apply the size constraint. This constraint checks whether the sum of the sizes of both files F_i and F_j exceeds the data block size T . If this is the case, $Similarity(F_i, F_j)$ is set to 0.
- **Calculating Cosine similarity:** After checking the size constraint, the similarity value is calculated for the remaining pairs of files F_i and F_j whose sum of sizes does not exceed T . This is performed using Formula 2:

$$Similarity(F_i, F_j) = \frac{\vec{F}_i \cdot \vec{F}_j}{\|\vec{F}_i\| \|\vec{F}_j\|} \quad (2)$$

where $\vec{F}_i$ (*resp.* $\vec{F}_j$) represents the weight vector associated to F_i (*resp.* F_j) such that:

- $\vec{F}_i \cdot \vec{F}_j = \sum_{k=1}^m Weight(F_i, Cl_k) \cdot Weight(F_j, Cl_k)$ where m is the total number of requesting clients.
- $\|\vec{F}_i\| = \sqrt{\sum_{k=1}^m Weight(F_i, Cl_k)^2}$
- $\|\vec{F}_j\| = \sqrt{\sum_{k=1}^m Weight(F_j, Cl_k)^2}$

It is important to note that the similarity value for a file F_i to itself equals to 1 ($Similarity(F_i, F_i) = 1$).

In our strategy, each file is associated with a vector of positive weights. Consequently, the similarity metric, bounded between 0 and 1, quantifies the relationship between files as follows: a value close to 0 indicates that the files are largely independent, *i.e.*, requested by almost entirely distinct sets of clients, whereas a value close to 1 indicates a strong correlation, *i.e.*, the files tend to share common access patterns. Consequently, as the similarity value approaches to 1, the more correlated are the associated files. The significance of the Cosine similarity metric lies in its ability to quantify the similarity between vectors irrespective of their magnitudes. Thus, the Cosine similarity helps to identify files with similar characteristics.

Since $Similarity(F_i, F_j) = Similarity(F_j, F_i)$, the similarity matrix is an upper triangular matrix of size $n \times n$ with n the number of files, in the form shown in Figure 4. Algorithm 2 details the generation steps of the similarity matrix.

$$SM = \begin{pmatrix} 1 & \text{Similarity}(F_1, F_2) & \cdots & \text{Similarity}(F_1, F_n) \\ & 1 & \cdots & \text{Similarity}(F_2, F_n) \\ & & \ddots & \vdots \\ & & & 1 & \text{Similarity}(F_{n-1}, F_n) \\ & & & & 1 \end{pmatrix}$$

Fig. 4: Structure of SM

Algorithm 2: Generation of the similarity matrix MS**Data:** Weight matrix WM, data block size T **Result:** Similarity matrix SM**Begin**

```

1 for each file  $F_i$  in WM do
2   for each file  $F_j$  in WM do
3     if  $F_i == F_j$  then
4       SM[i][j]  $\leftarrow$  1 ;
5     else if  $\text{Size}(F_i) + \text{Size}(F_j) > T$  then
6       SM[i][j]  $\leftarrow$  0 ;
7     else
8       SM[i][j]  $\leftarrow \frac{\sum_{k=1}^m WM[i][k] \cdot WM[j][k]}{\sqrt{\sum_{k=1}^m WM[i][k]^2} \sqrt{\sum_{k=1}^m WM[j][k]^2}}$  ;
End

```

3.4 Correlated Small Files Merging

In this step, we apply the hierarchical ascending clustering to group small files into clusters. Each cluster contains a set of correlated small files that are merged into a single large file. Hierarchical clustering algorithms organize data into a hierarchical structure through an iterative process based on a proximity matrix [74]. In our context, the similarity matrix computed in the previous step serves as this proximity matrix. The fundamental principle of hierarchical agglomerative clustering is to progressively group the most similar pairs of objects, prioritizing those with stronger mutual similarity over those that are more distant. The choice of hierarchical clustering is motivated by the fact that the number of clusters is not known in advance. Furthermore, such method can give multiple consistent partitions of small files at different levels for the same dataset without rerunning clustering [58, 67].

Initially, each small file is considered as an individual cluster. The algorithm then iteratively groups the two closest clusters, *i.e.*, those with the highest similarity value. This process continues by progressively combining the most correlated clusters until no further grouping is possible. This is detailed in the following steps:

- **Initializing clusters:** Initially, each small file is considered as a cluster. The initial clusters are then noted $\{C_1, C_2, \dots, C_n\}$, where each cluster C_i contains the small file F_i .
- **Stopping criterion:** All the similarity values in the upper triangular matrix SM are 0. In other words, the process is stopped when it is not possible to further group clusters. The reason is that the sum of sizes of the associated files exceeds the data block size T .
- **Grouping process:** This process is composed of the following steps and is repeated until the stopping criterion is reached:
 - **Identification of the highest similarity value:** The clusters pair having the highest similarity value from the similarity matrix SM is identified.
 - **Grouping clusters and updating the previous similarity matrix:** The clusters C_x and C_y identified in the previous step are grouped into a new cluster $C_{x,y}$ containing the files of C_x and those of C_y . The similarity matrix is then updated by only computing the similarity value of $C_{x,y}$ with the remaining clusters C_k . Two cases are distinguished:
 - If the total size of files of $C_{x,y}$ and those of C_k exceeds T , then $\text{Similarity}(C_{x,y}, C_k) = 0$.

- Otherwise, the similarity value is computed using Formula 3, as follows:

$$\text{Similarity}(C_{x,y}, C_k) = \max\{\text{Similarity}(F_i, F_j), \text{ where } F_i \in C_{x,y} \text{ and } F_j \in C_k\} \quad (3)$$

- **Final clustering result:** At the process end, we obtain as many large files as final clusters. Each cluster contains a group of correlated small files whose total size does not exceed T .

3.5 Large Files Storage in HDFS

Once the clusters are determined, a *part_** file is created for each cluster to store the associated small files. These *part_** files are then stored in the DataNodes. Additionally, an index file is created in the NameNode to store the metadata of each small file, as outlined below:

- *File name:* the name of the small file,
- *Large file name:* the name of the large file (*part_1*, *part_2*, etc.) containing the small file,
- *Offset:* the position of the small file into the *part_** file,
- *Size:* the size of the small file.

4 Illustrative Example

In this example, we assume having 12 small files whose sizes are given in Table 3 and the data block size is equal to 128 Mo.

File	Size in Mo
F ₁	20
F ₂	25
F ₃	30
F ₄	20
F ₅	15
F ₆	25
F ₇	10
F ₈	20
F ₉	30
F ₁₀	20
F ₁₁	25
F ₁₂	15

Table 3: Size of different small files

4.1 Frequency Matrix Generation

Table 4 depicts the frequency matrix used in this example.

	$TotalClients(F_i)$	Cl_1	Cl_2	Cl_3	Cl_4	Cl_5	Cl_6
F_1	4	4	0	3	0	2	1
F_2	5	3	4	0	2	5	2
F_3	3	0	3	0	2	0	1
F_4	6	5	4	3	2	1	4
F_5	4	2	0	5	0	3	4
F_6	2	1	0	0	0	0	1
F_7	3	2	0	3	0	0	4
F_8	3	0	2	0	3	0	2
F_9	1	0	2	0	0	0	0
F_{10}	2	0	2	0	3	0	0
F_{11}	4	2	1	0	3	0	4
F_{12}	3	1	1	0	0	0	1

Table 4: Number of requests of each file by each requesting client

4.2 Weight Matrix Generation

Having as input the FM given in Table 4, the Weight Matrix (WM) is generated using Formula 1. We present here how the weight vector associated to F_1 is computed:

- $TotalClients = 6$
- $TotalClients(F_1) = 5$
- $\log(\frac{TotalClients}{TotalClients(F_1)}) = \log(\frac{6}{5}) = 0.1761$
- The weight vector of F_1 : $\vec{F}_1 = \begin{pmatrix} Weight(F_1, Cl_1) = 4.70 \\ Weight(F_1, Cl_2) = 0.00 \\ Weight(F_1, Cl_3) = 3.52 \\ Weight(F_1, Cl_4) = 0.00 \\ Weight(F_1, Cl_5) = 2.35 \\ Weight(F_1, Cl_6) = 1.17 \end{pmatrix}$

Table 5 illustrates the obtained WM.

	Cl_1	Cl_2	Cl_3	Cl_4	Cl_5	Cl_6
F_1	4.70	0	3.52	0	2.35	1.17
F_2	3.52	4.70	0	2.35	5.88	2.35
F_3	0	3.52	0	2.35	0	1.17
F_4	5.88	4.70	3.52	2.35	1.17	4.70
F_5	2.35	0	5.88	0	3.52	4.70
F_6	1.17	0	0	0	0	1.17
F_7	2.35	0	3.52	0	0	4.70
F_8	0	2.35	0	3.52	0	2.35
F_9	0	2.35	0	0	0	0
F_{10}	0	2.35	0	3.52	0	0
F_{11}	2.35	1.17	0	3.52	0	4.70
F_{12}	1.17	1.17	0	0	0	1.17

Table 5: Obtained Weight Matrix (WM)

4.3 Similarity Matrix Generation

The similarity value between each pair files is then computed using Formula 2. For example, the sum size of F_1 and F_2 is $45 \text{ Mo} \leq 128 \text{ Mo}$. Thus, the similarity value between F_1 and F_2 is calculated as follows:

- $\vec{F}_1 \cdot \vec{F}_2 = (4.70 \times 3.52) + (0 \times 4.70) + (3.52 \times 0) + (0 \times 2.35) + (2.35 \times 5.88) + (1.17 \times 2.35) = 28.63$
- $\|\vec{F}_1\| = \sqrt{4.70^2 + 0^2 + 3.52^2 + 0^2 + 2.35^2 + 1.17^2} = 6.21$
- $\|\vec{F}_2\| = \sqrt{3.52^2 + 4.70^2 + 0^2 + 2.35^2 + 5.88^2 + 2.35^2} = 8.50$

Thus, $\text{Similarity}(F_1, F_2) = 0.55$. Table 6 shows the obtained similarity matrix.

	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}
F_1	1.00	0.55	0.45	0.682	0.34	0.125	0.55	0.25	0.00	0.00	0.333	0.15
F_2		1.00	0.33	0.49	0.68	0.29	0.24	0.15	0.13	0.08	0.40	0.12
F_3			1.00	0.47	0.24	0.14	0.66	0.19	0.08	0.18	0.50	0.27
F_4				1.00	0.35	0.15	0.57	0.28	0.11	0.22	0.55	0.34
F_5					1.00	0.28	0.40	0.20	0.12	0.14	0.48	0.18
F_6						1.00	0.24	0.19	0.09	0.08	0.36	0.12
F_7							1.00	0.28	0.13	0.21	0.46	0.26
F_8								1.00	0.16	0.12	0.33	0.14
F_9									1.00	0.11	0.22	0.09
F_{10}										1.00	0.26	0.11
F_{11}											1.00	0.34
F_{12}												1.00

Table 6: Obtained Similarity Matrix (SM).

4.4 Files Grouping

1. *Clusters initialization*: at first, each file represents a cluster. We then have 12 clusters as follows:

$$C_1 = \{F_1\}, C_2 = \{F_2\}, C_3 = \{F_3\}, C_4 = \{F_4\}, C_5 = \{F_5\}, C_6 = \{F_6\}, \\ C_7 = \{F_7\}, C_8 = \{F_8\}, C_9 = \{F_9\}, C_{10} = \{F_{10}\}, C_{11} = \{F_{11}\}, C_{12} = \{F_{12}\}$$

2. *Grouping process*:

- *Iteration 1*: F_1 and F_4 have the highest similarity value and are thus grouped in a new cluster $C_{1,4} = \{F_1, F_4\}$. The previous similarity matrix is then updated. For example, before computing the similarity value between $C_{1,4}$ and C_2 , the strategy checks whether the sum of file size of $C_{1,4}$ and C_2 exceeds or not the size of a block, 128 Mo in this example. Since, the sum is equal to 65 Mo, then the similarity value is computed and is equal to $\text{Similarity}(C_{1,4}, C_2) = \max\{\text{Similarity}(F_1, F_2), \text{Similarity}(F_4, F_2)\} = 0.55$. The new similarity matrix is depicted in Table 7. The next iterations alternate the same process of grouping the highest similar clusters and the update of the similarity matrix.
- *Iteration 2*: grouping C_2 and C_5 in a new cluster $C_{2,5} = \{F_2, F_5\}$.
- *Iteration 3*: grouping C_3 and C_7 in a new cluster $C_{3,7} = \{F_3, F_7\}$.
- *Iteration 4*: grouping $C_{1,4}$ and $C_{3,7}$ in a new cluster $C_{1,4,3,7} = \{F_1, F_3, F_4, F_7\}$.
- *Iteration 5*: grouping $C_{1,4,3,7}$ and $C_{2,5}$ in a new cluster $C_{1,2,3,4,5,7} = \{F_1, F_2, F_3, F_4, F_5, F_7\}$.
- *Iteration 6*: grouping C_6 and C_{11} in a new cluster $C_{6,11} = \{F_6, F_{11}\}$.
- *Iteration 7*: grouping $C_{6,11}$ and C_{12} in a new cluster $C_{6,11,12} = \{F_6, F_{11}, F_{12}\}$.
- *Iteration 8*: grouping $C_{6,11,12}$ and C_8 in a new cluster $C_{6,8,11,12} = \{F_6, F_8, F_{11}, F_{12}\}$.
- *Iteration 9*: grouping $C_{6,8,11,12}$ and C_{10} in a new cluster $C_{6,8,10,11,12} = \{F_6, F_8, F_{10}, F_{11}, F_{12}\}$.

	C _{1,4}	C ₂	C ₃	C ₅	C ₆	C ₇	C ₈	C ₉	C ₁₀	C ₁₁	C ₁₂
C _{1,4}	1.00	0.55	0.47	0.35	0.15	0.57	0.28	0.11	0.22	0.55	0.34
C ₂		1.00	0.33	0.68	0.29	0.24	0.15	0.13	0.08	0.40	0.12
C ₃			1.00	0.24	0.14	0.66	0.19	0.08	0.18	0.50	0.27
C ₅				1.00	0.28	0.4	0.2	0.12	0.14	0.48	0.18
C ₆					1.00	0.24	0.19	0.09	0.08	0.36	0.12
C ₇						1.00	0.28	0.13	0.21	0.46	0.26
C ₈							1.00	0.16	0.12	0.33	0.14
C ₉								1.00	0.11	0.22	0.09
C ₁₀									1.00	0.26	0.11
C ₁₁										1.00	0.34
C ₁₂											1.00

Table 7: New Similarity Matrix

3. *Final clusters*: at the end of the process, we obtain the following three clusters:

- **Cluster 1**: $C_{1,2,3,4,5,7} = \{F_1, F_2, F_3, F_4, F_5, F_7\}$, having 120 Mo as size.
- **Cluster 2**: $C_{6,8,10,11,12} = \{F_6, F_8, F_{10}, F_{11}, F_{12}\}$, having 105 Mo as size.
- **Cluster 3**: $C_9 = \{F_9\}$, having 30 Mo as size.

5 Experiments

5.1 Experimental Environment

We implemented a prototype system and conducted a comparative analysis with the following solutions:

- Native HDFS as a baseline solution,
- Hadoop MapFile [3] as a representative of Hadoop-based solutions,
- HPF [79] as a representative of indexation-based solutions, and,
- MAGC [69] as a representative of correlation-based solutions.

Comparison of the proposed solution and those of the literature is carried out from different perspectives, as follows:

- Storage performance with respect to disk utilization, including:
 - Analysis of the number of obtained large files,
 - Comparison of the strategies performances with respect to the optimal data block number,
 - Focus on the average data blocks utilization.
- Metadata memory consumption of the NameNode,
- Evaluation of the time performance for storing obtained large files.

The experimental environment was configured with a cluster consisting of four nodes, with one node serving as the NameNode and the remaining three nodes forming the DataNodes. We utilize and configure a virtual machine in the Microsoft Azure cloud [5]. Each node is a server with two CPU cores of 2.60 GHz, 16 GB for RAM, and 127 GB Hard Disk. The nodes operating system is Microsoft Windows Server 2019 Datacenter. The experiment utilized the Hadoop 3.3.6 version and JDK 1.8.0 version. The replication factor is set to 3.

We conduct experiments on ten distinct datasets, with a variable number of text files. The distribution of small file sizes used in our experiments is shown in Figure 5. The file sizes in each dataset range from 1.03KB to 9.99MB. Table 8 shows the total size of small files in each dataset as well as the minimum, maximum and average file size. Moreover, we take into account three different values for the data block size, namely 128 MB, 256 MB and 512 MB.

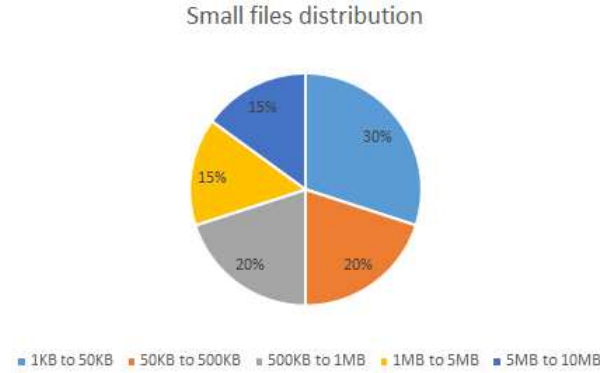


Fig. 5: Distribution of small file size

Dataset	# of files	Total size	Average File Size	Minimum File Size	Maximum File Size
Dataset1	1000	1.75 GB	1.79 MB	1.18 KB	9.69 MB
Dataset2	2000	3.52 GB	1.80 MB	1.03 KB	9.96 MB
Dataset3	3000	5.17 GB	1.76 MB	1.11 KB	9.99 MB
Dataset4	4000	6.94 GB	1.77 MB	1.07 KB	9.98 MB
Dataset5	5000	8.72 GB	1.78 MB	1.03 KB	9.98 MB
Dataset6	6000	10.40 GB	1.78 MB	1.03 KB	9.99 MB
Dataset7	7000	12.20 GB	1.78 MB	1.03 KB	9.99 MB
Dataset8	8000	13.80 GB	1.77 MB	1.03 KB	9.99 MB
Dataset9	9000	15.60 GB	1.78 MB	1.03 KB	9.99 MB
Dataset10	10000	17.40 GB	1.78 MB	1.03 KB	9.99 MB

Table 8: Datasets characteristics

5.2 DataNodes Storage Performance

5.2.1 Number of Obtained Large Files

Let us recall that the native HDFS uses as many data blocks as the number of small files, since storing each small file in a dedicated data block. This causes an extra disk usage of DataNodes. In order to further clarify this issue, if a dataset contains 10,000 small files, 30,000 data blocks are required in DataNodes for a replication factor set to 3.

Our strategy has two main objectives. First, it looks for a solution reducing the difference between the amount of data to be stored and the size required for storage on disk by reducing the number of necessary data blocks. This enhances file management operations within the NameNode. Additionally, CHAC is designed to generate clusters, each consisting of a group of correlated small files that are combined into a single large file, which is then stored in a data block. Consequently, the number of obtained clusters by CHAC is then equal to the number of large files resulting from small files merging process. Hereafter, *large file* and *data block* are used interchangeably while analyzing the obtained results.

Figure 6, Figure 7 and Figure 8 highlight the number of obtained large files for different strategies with respectively a block size set to 128 MB, 256 MB and 512 MB. The file number ranges from 1,000 to 10,000. MapFile is not considered in this experiment since this strategy puts all small files in a same data file.

By examining the obtained results, we remark that our strategy generates the minimum number of large files in all cases. Both strategies CHAC and HPF offer almost a closely number of data blocks. It is however important to emphasize that CHAC, unlike HPF, merges files while taking into account the similarity between them. This greatly improves the quality of the obtained large files through grouping small files of similar access pattern. Indeed, merged files are correlated and are almost used simultaneously by the requesting clients.

From the standpoint of metadata management, the system performance benefits significantly from an increase in block size. This improvement is intuitive, as larger blocks can encapsulate a greater number of small files, thereby

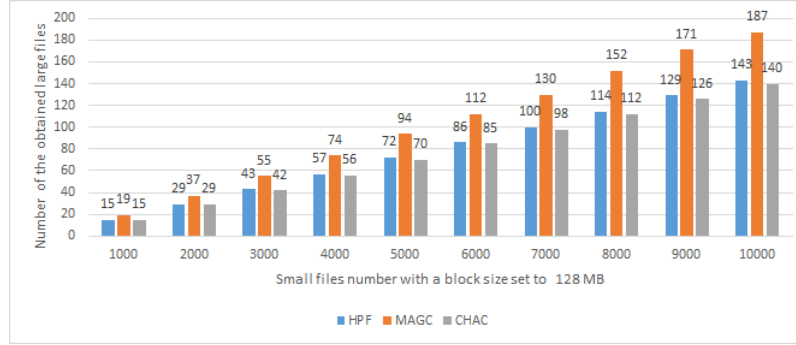


Fig. 6: Number of obtained large files with a block size set to 128 MB

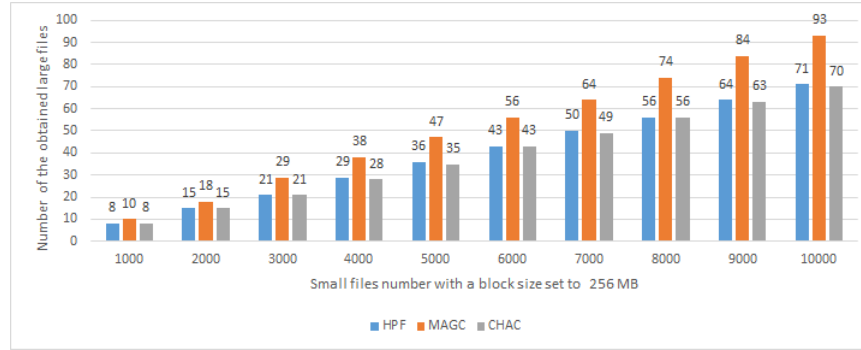


Fig. 7: Number of obtained large files with a block size set to 256 MB

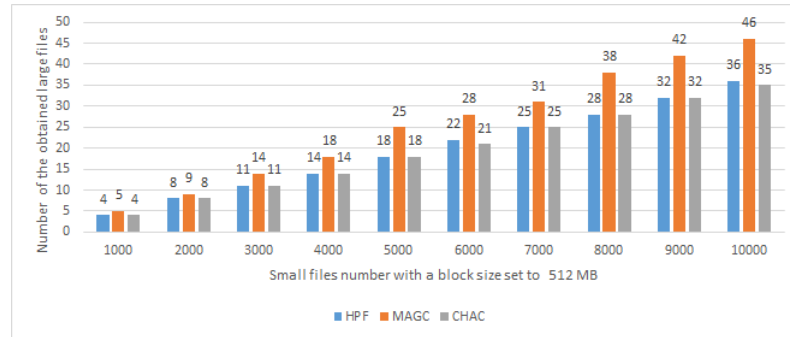


Fig. 8: Number of obtained large files with a block size set to 512 MB

reducing metadata overhead. Empirical results show that, for 10,000 small files, the number of resulting large files can decrease by approximately 70%. For example, under our proposed strategy, increasing the block size from 128 MB to 512 MB for 10,000 files reduces the number of obtained large files from 140 to 35, representing a reduction of approximately 75%.

On its side, MAGC typically requires minimum 20% of additional large files compared to CHAC. Although both CHAC and MAGC use the hierarchical ascending clustering, the number of obtained clusters is lower for our strategy. The obtained results prove the effective use of the criteria considered in CHAC, in addition to file size, as well as the utility of the Cosine similarity measure adopted as a shrewd tool for grouping correlated small files.

The effectiveness of the Cosine similarity compared to the Euclidean distance relied on in MAGC has also been proved in the literature [15].

However, it is important to note that increasing the data block size may negatively affect the quality of the resulting groups of correlated small files. In this regard, larger blocks can lead to less coherent clustering. In its current version, CHAC primarily focuses on maximizing the utilization of each data block's storage capacity. As a result, during the grouping process, a file may be included in a cluster even if it is not strongly correlated with the other files in that group. This important point is deeply analyzed in Section 6.

5.2.2 Optimal Data Block Number

The optimal block number refers to the ideal number of blocks to be used to store all small files contained in a given dataset. It is computed according to Formula 4, as follows:

$$Optimal_Block_Number = \left\lceil \frac{Dataset_Size}{T} \right\rceil \quad (4)$$

where:

- *Dataset_Size* represents the total size of small files.
- *T* represents the block size used in the HDFS.

Tables 9, 10 and 11 compare the MAGC, HPF and CHAC strategies based on the optimal number of blocks for different block size. These tables depict, for different numbers of small files (ranging from 1,000 to 10,000), the total file size, the optimal number of blocks, and the number of blocks required by each strategy. The "Best strategies" column indicates strategies that offer the number of blocks closest to the optimum. Finally, the "Number of excess blocks" column specifies the gap between the number of blocks used and the optimal number.

From the obtained results, our strategy is often the most successful, with a minimum number of exceed blocks. This is explained by the fact that our strategy tries to add the correlated files in the same large file without any constraint on the allowed correlation value. Indeed, in the current version of CHAC, no minimal correlation threshold is set. In other words, all correlations, regardless of their values, are considered valid for file grouping. It should also be noted that our strategy outperforms HPF and MAGC, especially when the block size is set to 128 MB. This suggests that the absence of a correlation threshold provides greater flexibility in merging process, which becomes particularly advantageous with smaller block sizes.

In the most cases, our strategy successfully identifies the ideal number of large files, and in the worst cases, the deviation is limited to just one. The stability of the obtained results demonstrates the scalability of our strategy, since it preserves clustering accuracy even when the dataset size grows.

5.2.3 Average Data Blocks Utilization

To evaluate space efficiency in HDFS, we can calculate the percentage of average data block utilization, which measures how efficiently storage blocks are used across the system. This metric, defined in Formula 5, indicates the proportion in percentage of allocated block space that is effectively used for storing actual small files:

$$Avg_Utilization = \frac{1}{DataBlocksNb} \sum_{i=1}^{DataBlocksNb} \frac{Used_Data_Block_Size(Data_Block_i)}{T} \times 100 \quad (5)$$

where:

- *DataBlocksNb* is the total number of data blocks storing merged small files.
- *Used_Data_Block_Size(Data_Block_i)* represents the actual size of data stored in the block containing the *i*-th obtained large file; that is, the portion of the block that is effectively used by the *i*-th large file.

# of files	Total size	Block size set to 128 MB				Best strategies	Number of excess blocks
		<i>Optimal_Block_Number</i>	MAGC	HPF	CHAC		
1000	1.75 GB	14	19	15	15	HPF, CHAC	1
2000	3.52 GB	29	37	29	29	HPF, CHAC	0
3000	5.17 GB	42	55	43	42	CHAC	0
4000	6.94 GB	56	74	57	56	CHAC	0
5000	8.72 GB	70	94	72	70	CHAC	0
6000	10.40 GB	84	112	86	85	CHAC	1
7000	12.20 GB	98	130	100	98	CHAC	0
8000	13.80 GB	111	152	114	112	CHAC	1
9000	15.60 GB	125	171	129	126	CHAC	1
10000	17.40 GB	140	187	143	140	CHAC	0

Table 9: Comparison between MAGC, HPF and CHAC strategies based on the optimal block number with a block size set to 128 MB

# of files	Total size	Block size set to 256 MB				Best strategies	Number of excess blocks
		<i>Optimal_Block_Number</i>	MAGC	HPF	CHAC		
1000	1.75 GB	7	10	8	8	HPF, CHAC	1
2000	3.52 GB	15	18	15	15	HPF, CHAC	0
3000	5.17 GB	21	29	21	21	HPF, CHAC	0
4000	6.94 GB	28	38	29	28	CHAC	0
5000	8.72 GB	35	47	36	35	CHAC	0
6000	10.40 GB	42	56	43	43	HPF, CHAC	1
7000	12.20 GB	49	64	50	49	CHAC	0
8000	13.80 GB	56	74	56	56	HPF, CHAC	0
9000	15.60 GB	63	84	64	63	CHAC	0
10000	17.40 GB	70	93	71	70	CHAC	0

Table 10: Comparison between MAGC, HPF and CHAC strategies based on the optimal block number with a block size set to 256 MB

# of files	Total size	Block size set to 512 MB				Best strategies	Number of excess blocks
		<i>Optimal_Block_Number</i>	MAGC	HPF	CHAC		
1000	1.75 GB	4	5	4	4	HPF, CHAC	0
2000	3.52 GB	8	9	8	8	HPF, CHAC	0
3000	5.17 GB	11	14	11	11	HPF, CHAC	0
4000	6.94 GB	14	18	14	14	HPF, CHAC	0
5000	8.72 GB	18	25	18	18	HPF, CHAC	0
6000	10.40 GB	21	28	22	21	CHAC	0
7000	12.20 GB	25	31	25	25	HPF, CHAC	0
8000	13.80 GB	28	38	28	28	HPF, CHAC	0
9000	15.60 GB	32	42	32	32	HPF, CHAC	0
10000	17.40 GB	35	46	36	35	CHAC	0

Table 11: Comparison between MAGC, HPF and CHAC strategies based on the optimal block number with a block size set to 512 MB

- T represents the block size used in the HDFS.

Since, $\sum_{i=1}^{DataBlocksNb} Used_Data_Block_Size(Data_Block_i)$ is equal the dataset size, Formula 5 can be expressed as follows:

$$Avg_Utilization = \frac{Dataset_Size}{DataBlocksNb \times T} \times 100 \quad (6)$$

According to the previous two sections dealing with the number of obtained data blocks, it is clear to note that the proposed solution offers interesting average data blocks utilization compared to the other solutions. Table 12 shows the obtained results by CHAC for the different data blocks sizes, 128MB, 256 MB, and 512 MB. These results confirm the efficiency of our approach in optimizing storage space usage of DataNodes. CHAC tries indeed to exploit the entire block space. This is performed while focusing on the quality of the obtained large files by considering additional criteria, improving then the processing task relying on these files.

Dataset size (in GB)	Block size set to 128 MB		Block size set to 256 MB		Block size set to 512 MB	
	DataBlocksNb	Avg_Utilization (in %)	DataBlocksNb	Avg_Utilization (in %)	DataBlocksNb	Avg_Utilization (in %)
1.75	15	93.33	8	87.50	4	87.50
3.52	29	97.10	15	93.87	8	88.00
5.17	42	98.48	21	98.48	11	94.00
6.94	56	99.14	28	99.14	14	99.14
8.72	70	99.66	35	99.66	18	96.89
10.4	85	97.88	43	96.74	21	99.05
12.2	98	99.59	49	99.59	25	97.60
13.8	112	98.57	56	98.57	28	98.57
15.6	126	99.05	63	99.05	32	97.50
17.4	140	99.43	70	99.43	35	99.43

Table 12: Average utilization of data blocks offered by CHAC

5.3 NameNode Memory Consumption

When a large number of files are stored individually in HDFS, the NameNode requires more memory to manage file metadata in the NameNode, leading to slower response time for file access. As a result, the NameNode can become a performance bottleneck in HDFS. By merging multiple small files into larger ones through the obtained clusters, the total number of managed files is significantly reduced, allowing the NameNode to operate more efficiently. In this respect, we examine the NameNode memory consumption by varying both the data block size and the files number.

Let us remember that the native HDFS uses as many data blocks as the number of small files, since storing each small file in a dedicated data block. This causes an overhead in the memory consumption of the NameNode. In order to further clarify this issue and based on [1], the HDFS namespace tree and associated metadata are maintained as objects in the NameNode's memory, each of which occupies approximately 150 bytes, as a rule of thumb. Thus, if a dataset contains 10,000 small files, the total memory required by the NameNode to store the metadata of these files is equal to: $150 \text{ bytes} \times (1 \text{ file inode} + (\text{number of blocks} \times \text{replication Factor})) = 150 \times (10,000 + (10,000 \times 3)) = 6,000,000 \text{ Bytes} \approx 5.72 \text{ GB}$.

Figure 9 (respectively 10 and 11) displays the NameNode memory requirement in bytes offered by CHAC compared to HPF, MapFile and MAGC with a block size set to 128 MB (respectively 256 MB and 512 MB).

An analysis of the experimental results reveals that, across all tested strategies, increasing the data block size leads to a reduction in NameNode memory usage. Noteworthy, MapFile requires significantly less metadata than the other strategies as it consists of only two components: a data file and an index file. Furthermore, the data

file is highly compressed. This result obtained by MapFile is also proven in [79]. Our proposed CHAC strategy demonstrates competitive performance in terms of NameNode memory efficiency compared to MAGC and HPF. Specifically, CHAC consumes approximately 55% less memory than HPF. This is, because HPF relies on two memory-intensive index structures, which are memory space intensive. Additionally, CHAC uses about 20% less memory than MAGC. This is because MAGC produces a larger number of merged large files, which increases the number of entries that the NameNode has to manage. In contrast, CHAC generates fewer, more optimized clusters, thereby reducing metadata overhead.

Given that the metadata memory consumption is inversely proportional to NameNode performance, relying on CHAC helps to mitigate the challenges associated with managing large volumes of small files. In fact, the system performance remains stable even when the block size varies across different numbers and sizes of small files.

In addition, from the obtained results, we observe that MapFile has a flatter curve with slower linear growth, while HPF has a steeper curve, indicating faster growth. MAGC and CHAC exhibit intermediate curves, showing moderate growth. CHAC, with its moderate growth rate, ensures scalability by efficiently handling the increasing load without causing significant performance degradation. Relying on CHAC offers then less negative performance impacts caused by small files, *i.e.*, the performance of the system can remain stable for files of various size distributions.

5.4 Storage Time of Obtained Large Files

The storage time represents the time taken by each strategy to store the obtained large files after merging. We evaluate the storage time in *ms* taken by CHAC compared to HPF, MapFile and MAGC with a block size set to 128 MB, 256 MB and 512 MB. The collected results are displayed in Figure 12, Figure 13 and Figure 14.

From the obtained results, HPF shows a slow and constant increase in storage time with respect to the number of files. MAGC illustrates a faster increase in storage time. CHAC shows a similar storage time to MAGC and HPF at first, but diverges slightly later, remaining below MAGC and MapFile. MapFile displays the highest storage time, with a significantly faster increase than other solutions as the number of files increases.

HPF achieves the shortest storage time among all evaluated strategies, primarily due to its file-level compression mechanism. However, despite this advantage, our proposed strategy CHAC consistently outperforms both MapFile and MAGC across all tested scenarios. This superior performance is mainly attributed to the fact that CHAC generates a minimal number of large files, thereby reducing the overhead associated with file creation and metadata handling. These results confirm the effectiveness of our approach in merging small files efficiently, while maintaining competitive storage performance. Moreover, CHAC offers a good trade-off between storage time and metadata optimization, making it well-suited in HDFS.

The purpose of the next section is to highlight the significance of the results obtained by the proposed CHAC strategy from the viewpoint of the quality of obtained large files. For this purpose, we conduct a detailed study of the quality of the clusters obtained.

6 Clusters Quality Evaluation

A clustering-quality measure is a function that, given a dataset and its partition into clusters, evaluates the quality of obtained clusters. Several metrics are proposed to assess the quality of clusters formed by considering the similarity within clusters and the dissimilarity between clusters such as intra-cluster inertia [44], C-index [36], Silhouette index [62], Davies–Bouldin index [24] and Calinski-Harabasz index [19].

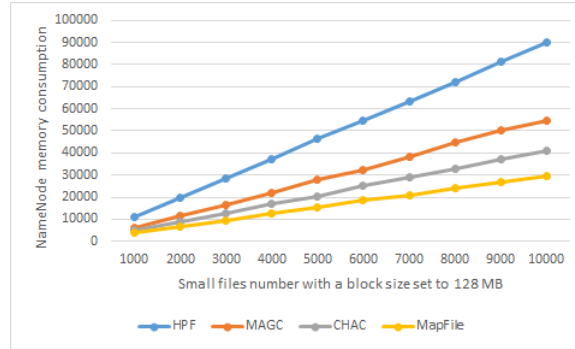


Fig. 9: NameNode memory consumption in bytes with a block size set to 128 MB

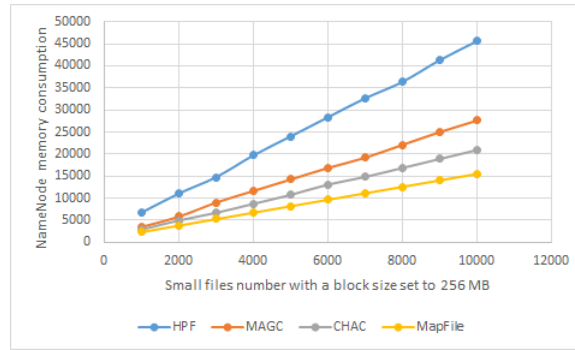


Fig. 10: NameNode memory consumption in bytes with a block size set to 256 MB

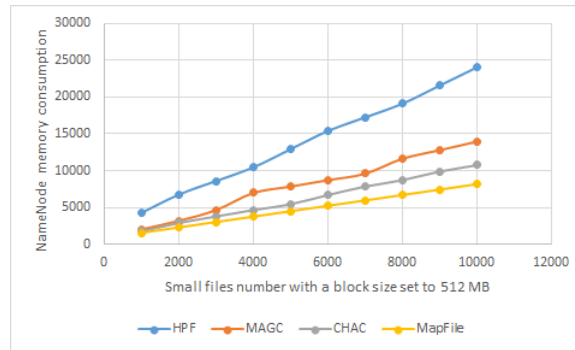
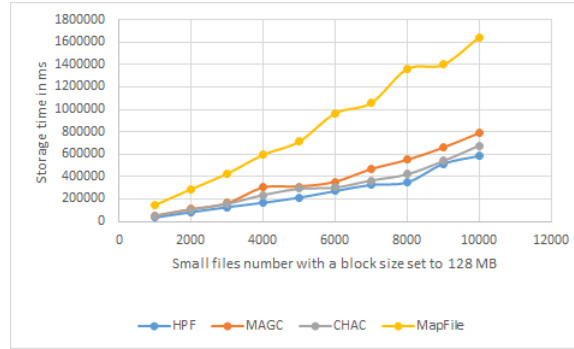
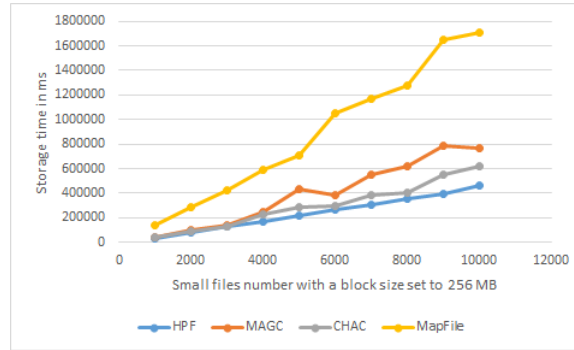
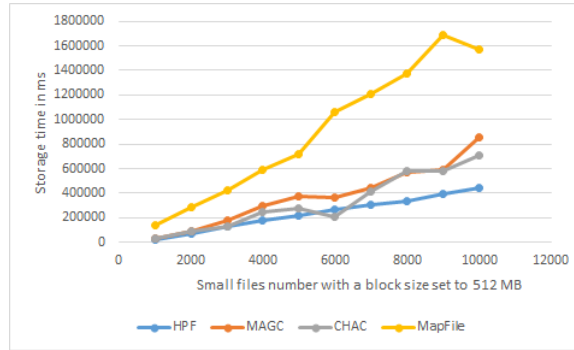


Fig. 11: NameNode memory consumption in bytes with a block size set to 512 MB

6.1 Distribution of Small File Weights

Figure 15, Figure 16 and Figure 17 show a few examples of scatter plots of datasets containing respectively 1,000, 2,000 and 3,000 files. In these plots, each file is characterized by its weight vector.

Let us recall that in our case, each obtained cluster is composed by a set of small files, merged into a large one. In this respect, our quality evaluation mainly focuses on the intra-cluster compactness. Indeed, our main concern is to study the quality of each obtained large file from the viewpoint of correlation between its corresponding small files.

Fig. 12: Storage time in *ms* with a block size set to 128 MBFig. 13: Storage time in *ms* with a block size set to 256 MBFig. 14: Storage time in *ms* with a block size set to 512 MB

The cluster compactness represents the cohesion of objects in an individual cluster. Cohesion is a measurement criteria used to determine that how well the objects of a dataset are combined together to form the good quality clusters [52, 78]. The main aim of the cohesion measurement is to determine the intra-cluster distance, which is the degree of association between the objects of a dataset within the same cluster. A smaller value indicates that the objects are close.

We choose to evaluate the obtained clusters using two measures, namely intra-cluster inertia and C-index. We compare our strategy to MAGC [69] since both strategies use the hierarchical ascending clustering to group small files. Three different values for block size are considered, namely 128 MB, 256 MB and 512 MB.

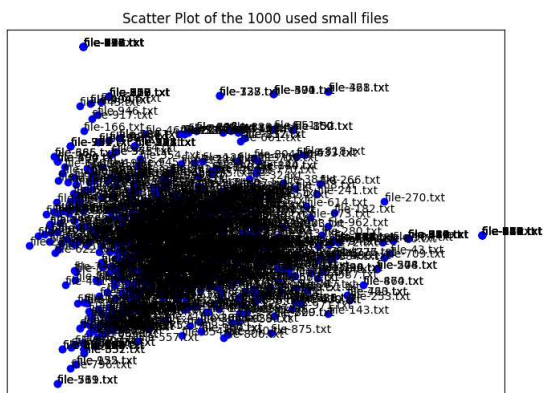


Fig. 15: Scatter plot of the dataset containing 1,000 small files

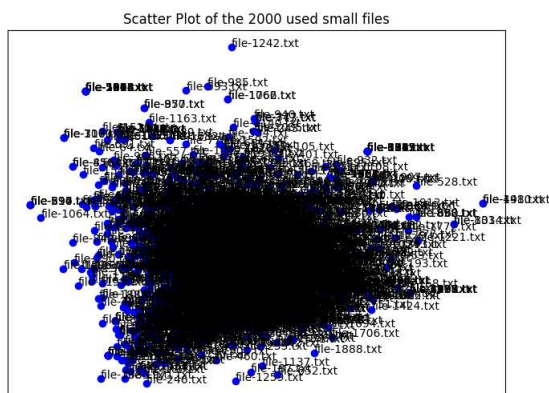


Fig. 16: Scatter plot of the dataset containing 2,000 small files

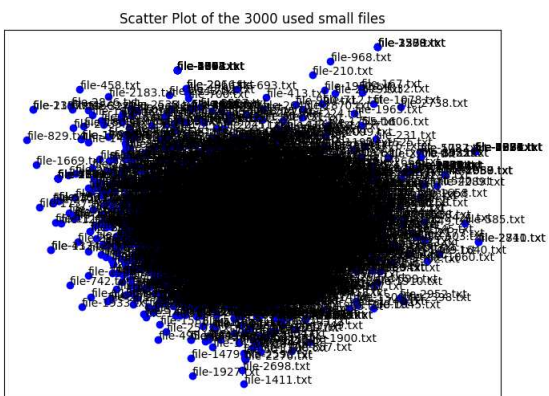


Fig. 17: Scatter plot of the dataset containing 3,000 small files

6.2 Quality Analysis using Intra-Cluster Inertia and C-index

Intra-cluster inertia evaluates the degree of homogeneity between the data associated with a cluster [43]. It is computed as the distance between all data instances within a given cluster and its centroid. A small value indicates a good quality clusters partitioning. Intra-cluster inertia for a cluster C_k is defined as given by Formula 7:

$$Intra_clust_dist(C_k) = \frac{1}{|C_k|} \sum_{F_i \in C_k} \|F_i - \mu_k\|^2 \quad (7)$$

where:

- μ_k is the centroid of the cluster C_k .
- $|C_k|$ is the total number of small files in the cluster C_k .
- $\|F_i - \mu_k\|$ is the distance between a file $F_i \in C_k$ and μ_k .

For a given dataset and a given size of a block, and since the number of the obtained clusters is high, we present in the depicted results the average of the intra-cluster inertia for the different clusters. This is performed according to Formula 8:

$$Avg_Intra_clust_dist = \frac{1}{K} \sum_{k=1}^K Intra_clust_dist(C_k) \quad (8)$$

where K is the total number of obtained clusters.

In order to go further in our analysis of the quality of the clusters, the C-index [36] measure is also used. For a given dataset, C-index allows the assessment of the cohesion of obtained clusters while considering the distribution of the whole set of objects (small files in our case). It is an internal cluster validity index and varies between 0 and 1. The lower is the value the better is the cluster partition. Formula 9 defines this measure:

$$C-index = \frac{S - S_{min}}{S_{max} - S_{min}} \quad (9)$$

where:

- S is the sum of the distances over all pairs of files from the same cluster. Let pNb be the number of pairs of files in the same cluster.
- S_{min} is the sum of the pNb smallest distances over all pairs of files in the dataset.
- S_{max} is the sum of the pNb largest distances over all pairs of files in the dataset.

Table 13 presents the results associated to the average intra-cluster inertia from experiments conducted on ten datasets, with a block size respectively equal to 128 MB, 256 MB and 512 MB. Table 14 depicts the C-index values obtained from the same experiments.

Based on the obtained results, it is evident that the clustering quality of our strategy outperforms that of MAGC. Specifically, the C-index value for CHAC is below 0.5, while the C-index value for MAGC exceeds 0.5. Since C-index assesses the compactness of clusters with lower values indicating better-defined and more cohesive clusters, these results clearly demonstrate the superiority of CHAC in forming high-quality clusters. Similarly, the average intra-cluster value of CHAC is always less than 0.5. This indicates that the clusters produced by CHAC are not only more compact but also group files that exhibit higher correlation with each other. This heightened correlation within clusters reflects a better alignment of grouped files in terms of similarity, which is a key objective in clustering tasks. Consequently, this suggests that employing additional criteria (file size, requests number and requesting clients) constitutes a relevant and valuable approach.

From the viewpoint of block size, the gain in C-index reaches up to 52% in certain configurations (*e.g.*, 6,000 files with a block size set to 128 MB). Overall, the best performance is observed with 128 MB, suggesting that

# of files	Block size set to 128 MB			Block size set to 256 MB			Block size set to 512 MB		
	MAGC	CHAC	Gain (in %)	MAGC	CHAC	Gain (in %)	MAGC	CHAC	Gain (in %)
1000	0.507	0.162	68.02	0.506	0.280	44.65	0.503	0.451	10.23
2000	0.548	0.277	49.38	0.547	0.358	34.61	0.551	0.462	16.14
3000	0.566	0.279	50.23	0.561	0.364	35.23	0.564	0.462	18.14
4000	0.567	0.352	37.88	0.568	0.426	24.95	0.567	0.493	13.08
5000	0.578	0.409	29.25	0.577	0.475	17.70	0.577	0.519	9.95
6000	0.541	0.191	64.64	0.539	0.260	51.85	0.540	0.360	33.24
7000	0.547	0.222	59.35	0.527	0.276	47.68	0.564	0.390	30.90
8000	0.523	0.189	63.92	0.523	0.257	50.96	0.524	0.369	29.56
9000	0.537	0.176	67.29	0.540	0.236	56.37	0.521	0.358	31.36
10000	0.556	0.171	69.20	0.531	0.237	55.44	0.523	0.361	31.01

Table 13: Cluster compactness evaluation using average intra-cluster inertia

# of files	Block size set to 128 MB			Block size set to 256 MB			Block size set to 512 MB		
	MAGC	CHAC	Gain (in %)	MAGC	CHAC	Gain (in %)	MAGC	CHAC	Gain (in %)
1000	0.602	0.325	46.03	0.602	0.443	26.50	0.604	0.503	16.74
2000	0.565	0.400	29.21	0.566	0.443	21.78	0.565	0.488	13.68
3000	0.559	0.400	28.42	0.561	0.438	21.84	0.560	0.476	15.13
4000	0.512	0.379	25.96	0.508	0.412	18.84	0.509	0.414	18.73
5000	0.472	0.367	22.20	0.471	0.390	17.22	0.471	0.410	12.97
6000	0.574	0.274	52.25	0.735	0.381	48.09	0.573	0.419	26.98
7000	0.557	0.276	50.50	0.639	0.380	40.52	0.570	0.420	26.32
8000	0.542	0.271	49.98	0.623	0.377	39.58	0.563	0.416	26.23
9000	0.541	0.270	50.10	0.699	0.380	45.63	0.579	0.411	28.97
10000	0.524	0.279	46.75	0.722	0.372	48.46	0.596	0.419	26.36

Table 14: Cluster compactness evaluation using C-index

smaller block sizes lead to better cluster compactness. The average intra-cluster inertia of CHAC approaches to 0 when the block size is 128 MB. The gains in the average intra-cluster value range from 9% to 69%, with the best values again obtained for a block size set to 128 MB.

The best results in terms of both clustering quality and gain are achieved with a 128 MB block size, which confirms that CHAC's criteria are particularly effective at this level of granularity. This means that the clustering is effective when the block size is reduced. Indeed, a block with a small size will hold a group of small files with a high similarity values. However, if the block size is large, the number of merged small files is higher. In this case, a block will contain a group of small files with reduced similarity values. For larger blocks (256 MB and 512 MB), the clustering quality decreases, yet CHAC consistently remains more effective than MAGC. In this regard, setting a minimum threshold for average correlation among files within a block could provide a compromise between compactness (*i.e.*, limiting the number of resulting large files) and clustering quality (*i.e.*, ensuring a minimum degree of correlation among files within each block).

From the small files number viewpoint, we observe that CHAC achieves a significant performance improvement as far as the size of the dataset increases. Thus, even when the number of files increases (up to 6,000), the scalability of CHAC maintains significant performance gains, especially with small to medium block sizes. This proves the ability of CHAC to manage and organize a growing number of files into cohesive and compact clusters, without degradation in clustering quality. This further reinforces the significance of our criteria in the merging process, highlighting their crucial role in achieving better clustering results. These criteria are used in the merging process to achieve a more accurate representation of file usage patterns and relationships, resulting in better cluster cohesion and overall efficiency. Thus, the improvement in clustering quality with a larger dataset highlights the relevance and effectiveness of using multiple criteria during the merging process.

7 Conclusion and Future Works

The performance of HDFS dramatically degrades when dealing with large amounts of small files. This indeed results in a high memory consumption of NameNode and serious access latency issues. In this paper, we present a review of various research solutions dealing with the small files problem in HDFS. We then propose CHAC, a new strategy for merging small files. Our strategy takes into account several parameters to select which small files need to be merged together into the same large file. The Cosine similarity is then used as a basis for determining correlated files. Next, an algorithm for hierarchical ascending clustering is relied on towards determining the groups of correlated small files while taking care of the size constraint of a data block. Experimental results show that CHAC not only ensures a smaller number of obtained large files, but also improves the memory consumption of the NameNode. Our strategy also minimizes the storage time taken to store the obtained large files and optimizes the storage performance of DataNodes as well. We use two clustering-quality measures to assess the quality of the obtained large files after merging.

As part of future work, on the one hand, the work proposed here constitutes a key step towards proposing new instances of the generic process we proposed through several aspects, as follows:

- First, other measures for file weighting can be considered, while looking for supplementary features of small files.
- Second, correlation assessment using other formulas offers a good research opportunity to rely simultaneously on user access pattern but also semantic correlation between files.
- Third, some other ways for constructing groups of correlated small files can also be investigated, like high utility pattern mining as well as graph clustering.
- Finally, further studies of constraints can be introduced in order to offer a trade-off between a compact number of obtained large files and a high quality of the obtained groups from the viewpoint of correlation.

On the other hand, it will be interesting to add some components to the proposed solution, mainly covering the following points:

- Designing a compression technique at the data block level constitutes an important issue to be considered.
- Adding a cache technique [77] to improve access to small files is an interesting complementary part to our proposal.
- Coupling our solution with a placement strategy of the obtained large files taking into consideration available data blocks and data balance [28] is an important issue. In this respect, the replication factor of obtained large files can be dynamically assessed according to the characteristics of the associated merged small files.
- Extending our work to be able to take into consideration file deleting and updating as well as adding new small files in an incremental manner is worthwhile towards covering applications where a data stream is processed.
- Applying processing tools like MapReduce and Spark on the obtained large files offered by our solution also constitutes an important piece of work to be performed.

From a practical viewpoint, since datasets in our experiments are composed of text files, other file formats and datasets [47] will be considered. Indeed, the “how” and “why” questions about the success or the failure of a solution on a given dataset still need answers that are more informative.

References

1. Cloudera: <https://www.cloudera.com/blog/technical/small-files-big-foils-addressing-the-associated-metadata-and-application-challenges.html>. Accessed on May 20th, 2025.
2. Hadoop Archive (HAR): <https://Hadoop.apache.org/docs/stable/Hadoop-archives/HadoopArchives.html>. Accessed on May 20th, 2025.
3. Hadoop MapFile: <https://Hadoop.apache.org/docs/r2.7.5/api/org/apache/Hadoop/io/MapFile.html>. Accessed on May 20th, 2025.

4. Hadoop SequenceFile: <https://Hadoop.apache.org/docs/stable/api/org/apache/Hadoop/io/SequenceFile.html>. Accessed on May 20th, 2025.
5. Microsoft Azure: <https://azure.microsoft.com/>. Accessed on May 20th, 2025.
6. Raveena Aggarwal, Jyoti Verma, and Manvi Siwach. Small files' problem in Hadoop: A systematic literature review. *Journal of King Saud University - Computer and Information Sciences*, 34(10, Part A):8658–8674, 2022.
7. Mohd Abdul Ahad and Ranjit Biswas. Dynamic merging based small file storage (DM-SFS) architecture for efficiently storing small size files in Hadoop. In *Proceedings of the International Conference on Computational Intelligence and Data Science*, pages 1626–1635, 2018.
8. Mohammad Al-Masadeh and Fahad Al-Zahrani. The role of vertical elastic NameNode in handling big data in small files. *Arabian Journal for Science and Engineering*, 47:1609–1617, 2021.
9. Mohammad Bahjat Al-Masadeh, Mohd Sanusi Azmi, and Sharifah Sakinah Syed Ahmad. Tiny datablock in saving Hadoop distributed file system wasted memory. *International Journal of Electrical and Computer Engineering*.
10. Neeta Alange and Anjali Anjali. Optimization of small sized file access efficiency in Hadoop distributed file system by integrating virtual file system layer. *International Journal of Advanced Computer Science and Applications*, 13(6), 2022.
11. Neeta Alange and P. Vidya Sagar. Small files access efficiency in Hadoop distributed file system a case study performed on British library text files. *Cluster Computing*, 26(6):3381–3388, 2023.
12. Adnan Ali, Nada Masood Mirza, Rawad Bader, and Mohamad Khairi Ishak. A simple approach for data cleansing on Hadoop framework using file merging technique. In *2022 Ninth International Conference on Software Defined Systems (SDS)*, pages 1–5, 2022.
13. Adnan Ali, Nada Masood Mirza, and Mohamad Khairi Ishak. Enhanced best fit algorithm for merging small files. *Computer Systems Science and Engineering*, 46(1):913–928, 2023.
14. Afzal Badshah, Ali Daud, Riad Alharbey, Ameen Banjar, Amal Bukhari, and Bader Alshemaimri. Big data applications: overview, challenges and future. *Artificial Intelligence Review*, 57(11):290, 2024.
15. Rashid Behzadidoost and Habib Izadkhan. Identifying effective algorithms and measures for enhanced clustering quality: A comprehensive examination of arbitrary decisions in hierarchical clustering algorithms. *Journal of Classification*, <https://doi.org/10.1007/s00357-025-09506-5>.
16. Sachin Bende and Rajashree Shedge. Dealing with small files problem in Hadoop distributed file system. *Procedia Computer Science*, 79:1001–1012, 2016. Proceedings of International Conference on Communication, Computing and Virtualization (ICCCV).
17. Kyoungsoo Bok, Hyunkyo Oh, Jongtae Lim, Yosop Pae, Hyoungrak Choi, Byoungyup Lee, and Jaesoo Yoo. An efficient distributed caching for accessing small files in HDFS. *Cluster Computing*, 20:3579–3592, 2017.
18. Xun Cai, Cai Chen, and Yi Liang. An optimization strategy of massive small files storage based on HDFS. In *Proceedings of the Joint International Advanced Engineering and Technology Research Conference (JIAET)*, pages 225–230, 2018.
19. T. Caliński and J Harabasz. A dendrite method for cluster analysis. *Communications in Statistics - Theory and Methods*, 3:1–27, 1974.
20. Dingchao Chen, Chase Q. Wu, Wei Shen, and Yu Zhang. On a small file merger for fast access and modifiability of small files in HDFS. In *2021 IEEE/ACS 18th International Conference on Computer Systems and Applications (AICCSA)*, pages 1–8, 2021.
21. Xingyu Chen, Wenhao Zhu, and Ke Yan. Massive small file storage scheme based on association rule mining. In *Proceedings of the 3rd International Conference on Big Data Engineering*, pages 34–40, 2021.
22. Hanène Chettaoui and Farah Hkiri. Combining similarity-based correlation and hierarchical ascending clustering for small files problem in HDFS. In *Proceedings of the 39th International Conference on Advanced Information Networking and Applications (AINA-2025), Barcelona, Spain*, pages 234–244, 2025.
23. Chang Choi, Chulwoong Choi, Junho Choi, and Pankoo Kim. Improved performance optimization for massive small files in cloud computing environment. *Annals of Operations Research*, 265:305–317, 2018.
24. David L. Davies and Donald W. Bouldin. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1:224–227, 1979.
25. Dipayan Dev and Ripon Patgiri. HAR+: Archive and metadata distribution! Why not both? In *2015 International Conference on Computer Communication and Informatics (ICCCI)*, pages 1–6, 2015.
26. Bo Dong, Qinghua Zheng, Feng Tian, Kuo-Ming Chao, Rui Ma, and Rachid Anane. An optimized approach for storing and accessing small files on cloud storage. *Journal of Network and Computer Applications*, 35(6):1847–1862, 2012.
27. Passent M ElKafrawy, Amr M Sauber, and Mohamed M Hafez. HDFSX: Big data distributed file system with small files support. In *Proceedings of the 12th International Computer Engineering Conference (ICENCO)*, pages 131–135, 2016.
28. Rhauani Weber Aita Fazul, Odorico Machado Mendizabal, and Patrícia Pitthan Barcelos2. Analyzing the stability, efficiency, and cost of a dynamic data replica balancing architecture for HDFS. *Annals of Telecommunications*, <https://doi.org/10.1007/s12243-025-01093-1>, 2025.
29. Songling Fu, Ligang He, Chenlin Huang, Xiangke Liao, and Kenli Li. Performance optimization for managing massive numbers of small files in distributed file systems. *IEEE Transactions on Parallel and Distributed Systems*, 26(12):3433–3448, 2015.
30. Yongqiang Gao, Shunyi Feng, and Zhenkun Li. A distributed cache mechanism of HDFS to improve learning performance for deep reinforcement learning. In *Proceedings of the IEEE International Conference on Parallel and Distributed Processing with Applications, Big Data and Cloud Computing, Sustainable Computing and Communications, Social Computing and Networking (ISPA/BDCloud/SocialCom/SustainCom)*, pages 280–285, 2022.
31. Anja Gruenheid, Jesús Camacho-Rodríguez, Carlo Curino, Raghu Ramakrishnan, Stas Pak, Sumedh Sakdeo, Lenisha Gandhi, Sandeep Singhal, Pooja Nilangekar, and Daniel Abadi. AutoComp: Automated data compaction for log-structured tables in data lakes. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD/PODS)*, 2025.

32. Shaopeng Guan, Conghui Zhang, Yilin Wang, and Wenqing Liu. Hadoop-based secure storage solution for big data in cloud computing environment. *Digital Communications and Networks*, 10:227–236, 2024.
33. Venkat N. Gudivada, Dhana L. Rao, and Amogh R. Gudivada. Chapter 11 - Information Retrieval: Concepts, Models, and Systems. In *Computational Analysis and Understanding of Natural Languages: Principles, Methods and Applications*, volume 38 of *Handbook of Statistics*, pages 331–401. Elsevier, 2018.
34. Hui He, Zhonghui Du, Weizhe Zhang, and Allen Chen. Optimization strategy of Hadoop small file storage for big data in healthcare. *The Journal of Supercomputing*, 72:3696–3707, 2016.
35. Martin Hentschel, Jonathan Dees, Florian Funke, Max Heimel, and Ismail Oukid. Building a data management system for the cloud: Lessons learned and future directions. *Datenbank-Spektrum*, 25(1):17–28, 2025.
36. LJ Hubert and JR Levin. A general statistical framework for assessing categorical clustering in free recall. *Psychological Bulletin*, 83:1072–1080, 1976.
37. Fredrick Ishengoma. Efficient small file management in Hadoop distributed file system for enhanced e-government services. *Technological Sustainability*, 4(2):160–180, 2025.
38. Ishwarappa and J. Anuradha. A brief introduction on big data 5Vs characteristics and Hadoop technologie. In *Proceedings of the International Conference on Intelligent Computing, Communication and Convergence (ICCC)*, *Procedia Computer Science*, pages 319–324, 2015.
39. D. S. Jayalakshmi and R Srinivasan. A greedy file merging algorithm for handling small files in HDFS. *International Journal of Pure and Applied Mathematics*, 118(22):323–330, 2018.
40. Liu Jiang, Bing Li, and Meina Song. The optimization of HDFS based on small files. In *Proceedings of the 3rd IEEE International Conference on Broadband Network and Multimedia Technology (IC-BNMT)*, pages 912–915, 2010.
41. Chao Jin, David Abramson, Jake Carroll, Zhengchun Liu, and Rajkumar Kettimuthu. Moving small files in a networked environment. *Future Generation Computer Systems*, 139:167–180, 2023.
42. Jakub Kopec. Evaluating methods of transferring large datasets. In *Proceedings of the Supercomputing Frontiers Asia SCFA*, pages 102–120, 2022.
43. Jean-Charles Lamirel, Pascal Cuxac, Raghvendra Mall, and Ghada Safi. A new efficient and unbiased approach for clustering quality evaluation. In *New Frontiers in Applied Data Mining*, pages 209–220, 2012.
44. Ludovic Lebart, Alain Morineau, and Marie Piron. *Statistique exploratoire multidimensionnelle : Visualisation et inférence en fouille de données*, 4th edition, Dunod. 2006.
45. Xiuqiao Li, Bin Dong, Limin Xiao, Li Ruan, and Yi Ding. Small files problem in parallel file system. In *IEEE International Conference on Network Computing and Information Security (NCIS)*, pages 227–232, 2011.
46. Chengjian Liu, Pengcheng Shi, Yihong Li, Lixin Liang, and Lin Lin. Efficient data loading for deep neural network training. In *Proceedings of the 9th IEEE International Conference on Big Data Computing and Communications (BigCom)*, pages 211–218, 2023.
47. Guozhi Liu, Weiwei Lin, Haotong Zhang, Jianpeng Lin, Shaoliang Peng, and Keqin Li. Public datasets for cloud computing: A comprehensive survey. *ACM Computing Surveys*, 57(8):1–38, 2025.
48. Jun Liu. Storage-optimization method for massive small files of agricultural resources based on Hadoop. *Journal of Advanced Computational Intelligence and Intelligent Informatics*, 23(4):634–640, 2019.
49. Junnan Liu, Shengyi Jin, Dong Wang, and Han Li. An archive-based method for efficiently handling small file problems in HDFS. *Concurrency and Computation: Practice and Experience*, pages 1–15, 2024.
50. Grant Mackey, Saba Sehrish, and Jun Wang. Improving metadata management for small files in HDFS. In *IEEE International Conference on Cluster Computing and Workshops*, pages 1–4, 2009.
51. Salman Niazi, Mikael Ronström, Seif Haridi, and Jim Dowling. Size matters: Improving the performance of small files in Hadoop. In *Proceedings of the 19th International Middleware Conference*, pages 26–39, 2018.
52. Nisha and Puneet Jai Kaurt. Cluster quality based performance evaluation of hierarchical clustering method. In *2015 1st International Conference on Next Generation Computing Technologies (NGCT)*, pages 649–653, 2015.
53. Kwangil Park, June Seok Hong, and Wooju Kim. A methodology combining cosine similarity with classifier for text classification. *Applied Artificial Intelligence*, 34(5):396–411, 2020.
54. Junjie Peng, Danxu Liu, Yingtao Wang, Ying Zeng, Feng Cheng, and Wenqiang Zhang. Weight-based strategy for an I/O-intensive application at a cloud data center. *Concurrency and Computation: Practice and Experience*, 30(19), 2018.
55. Priyanka G. Phakade and Suhas Raut. An innovative strategy for improved processing of small files in Hadoop. *International Journal of Application or Innovation in Engineering and Management*, 3(7):278–280, 2014.
56. Foster Provost and Tom Fawcett. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media, 2013.
57. Yingjin Qian, Marc-André Vef, Patrick Farrell, Andreas Dilger, Xi Li, Shuichi Ihara, Yinjin Fu, Wei Xue, and André Brinkmann. Combining buffered I/O and direct I/O in distributed file systems. In *Proceedings of the 22nd USENIX Conference on File and Storage Technologies (FAST)*, pages 17–33, 2024.
58. Xingcheng Ran, Yue Xi, Yonggang Lu, Xiangwen Wang, and Zhenyu Lu. Comprehensive survey on hierarchical clustering algorithms and the recent developments. *Artificial Intelligence Review*, 56(8):8219–8264, 2023.
59. R. Rathidevi and R. Parameswari. CSFC: A new centroid based clustering method to improve the efficiency of storing and accessing small files in Hadoop. *International Journal of Recent Technology and Engineering*, 8(4S5):122–127, 2019.
60. Fuji Ren and Mohammad Golam Sohrab. Class-indexing-based term weighting for automatic text classification. *Information Science*, 236:109–125, 2013.

61. Thomas Renner, Johannes Müller, Lauritz Thamsen, and Odej Kao. Addressing Hadoop's small file problem with an appendable archive file format. In *Proceedings of the Computing Frontiers Conference*, pages 367–372, 2017.
62. Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.
63. Vijay Shankar Sharma, Asyraf Afthanorhan, Nemi Chand Barwar, Satyendra Singh, and Hasmat Malik. A dynamic repository approach for small file management with fast access time on Hadoop cluster: Hash Based Extended Hadoop Archive. *IEEE Access*, 10:36856–36867, 2022.
64. Isma Farah Siddiqui, Nawab Muhammad Faseeh Qureshi, Bhawani Shankar Chowdhry, and Muhammad Aslam Uqaili. Pseudo-cache-based IoT small files management framework in HDFS cluster. *Wireless Personal Communications*, 113(3):1495–1522, 2020.
65. J. Sirisha, K. Swathi, and P. Rama Koteswara Rao. An efficient approach for merging small files in HDFS storage and accessing small files by using name node method in big data. *Turkish Journal of Computer and Mathematics Education*, 12(10):4667–4673, 2021.
66. L. Tian, Y. Sun, and L. Yang. Overview of storage architecture and strategy of HDFS. *Applied Mathematics, Modeling and Computer Simulation*, 20:945–954, 2022.
67. Ping Tian, Huitao Shen, and Ahad Abolfathi. Towards efficient ensemble hierarchical clustering with MapReduce-based clusters clustering technique and the innovative similarity criterion. *Journal of Grid Computing*, 20(4):34, 2022.
68. Chatuporn Vorapongkitipun and Natawut Nupairoj. Improving performance of small-file accessing in Hadoop. In *2014 11th International Joint Conference on Computer Science and Software Engineering (JCSSE)*, pages 200–205, 2014.
69. Khin Su Su Wai, Julia Myint, and Tin Tin Yee. Merging small files based on agglomerative hierarchical clustering on HDFS for cloud storage. In *Proceedings of the 16th International Conference on Computer Applications (ICCA)*, pages 34–39, 2018.
70. Kun Wang, Yang Yang, Xuesong Qiu, and Zhipeng Gao. MOSM: An approach for efficient storing massive small files on Hadoop. In *2017 IEEE 2nd International Conference on Big Data Analysis (ICBDA)*, pages 397–401, 2017.
71. Miao Wang, Yanming Zhang, and Hualei Wang. SF-SNF: A small-file sniffer system for Hadoop clusters. In *IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*, pages 1663–1669, 2024.
72. Song Wang, Hai Lan, Yuwei Peng, and Zhiyong Peng. Consolidating industrial small files using robust graph clustering. *IEEE Transactions on Network Science and Engineering*, 10(5):2814–2831, 2023.
73. Wenjing Wu, Huiyi Liu, Liting Duan, and Siying Xu. SequenceFile storage optimization method based on pile structure. In *2021 IEEE International Conference on Artificial Intelligence and Computer Applications (ICAICA)*, pages 118–122, 2021.
74. Rui Xu and Donald Wunsch. Survey of clustering algorithms. *IEEE Transactions on Neural Networks*, 16(3):645–678, 2005.
75. Weijing Xu, Yitao Qiu, Congfeng Jiang, Junming Liu, Xiaoyu Wang, and Shijie Chen. Merging and cooperative caching based HDFS small files access performance optimization. In *2024 4th International Conference on Communication Technology and Information Technology (ICCTIT)*, pages 211–219, 2024.
76. Wentao Xu, Xin Zhao, Bin Lao, and Ge Nong. Enhancing HDFS with a full-text search system for massive small files. *The Journal of Supercomputing*, 77:7149–7170, 2021.
77. Weitao Yang, Li Pan, and Shijun Liu. Caching or not: An online cost optimization algorithm for geodistributed data analysis in cloud environments. *Journal of Network and Computer Applications*, 229:103942, 2024.
78. Meng Yuan, Justin Zobel, and Pauline Lin. Measurement of clustering effectiveness for document collections. *Information Retrieval Journal*, 25:239–268, 2022.
79. Yanlong Zhai, Jude Tchaye-Kondi, Kwei-Jay Lin, Liehuang Zhu, Wenjun Tao, Xiaojiang Du, and Mohsen Guizani. Hadoop Perfect File: A fast and memory-efficient metadata access archive file to face small files problem in HDFS. *Journal of Parallel and Distributed Computing*, 156:119–130, 2021.
80. Hong Zhang and Yangyang Chen. Multi-modal campus one-stop data storage optimization method for massive small files. In *2023 8th International Conference on Information Systems Engineering (ICISE)*, pages 338–341, 2023.
81. Jingyu Zhang, Jiachen Tian, and Lailong Luo. BTDFS: B+ tree-based distributed file storing mechanism for improving small file storage. In *Proceedings of the IEEE 24th International Conference on High Performance Computing and Communications; 8th Int Conf on Data Science and Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud and Big Data Systems and Application (HPCC/DSS/SmartCity/DependSys)*, pages 1000–1005, 2022.
82. Yizhi Zhang, Zhengdong Zhu, Honglin Cui, Xiaoshe Dong, and Heng Chen. Small files storing and computing optimization in Hadoop parallel rendering. *Concurrency and Computation: Practice and Experience*, 29:1–13, 2017.
83. Zongwei Zhu, Luchao Tan, Yinzheng Li, and Cheng Ji. PHDFS: Optimizing I/O performance of HDFS in deep learning cloud computing platform. *Journal of Systems Architecture*, 109:101810, 2020.